SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Curriculum

Major :

Multimodal Information Application

Graduation Internship Report

Named Entities Extraction from Unstructured Business Documents

---

Realized by :
Eya BAKLOUTI

Academic Supervisor :
Mr. Chiheb REBAI

Professional Supervisor :
Mr. Amir KRIFA

Work proposed and fulfilled in collaboration with :
EY



Academic year : 2020-2021

Higher School of Communications of Tunis
SUP'COM
Elghazala Technopark - Ariana 2083 - Tunisia
Tel. +216 70 240 900 – website : www.supcom.mincom.tn

Acknowledgements

It would be a great pleasure, before presenting my project, to express my gratitude to all those who supported me and contributed to the success of this project.

First, I want to express my profound gratitude to my wonderful family for always believing in me and for giving me the strength to be the person I am today.

My sincere thanks go to my supervisor **Mr. Amir Krifa**, Director Data Science & AI at Ernst & Young, for offering me the opportunity to develop and work on new skills as well as expanding my knowledge horizons and for his guidance and help throughout the internship.

A special thanks and appreciation go to my school supervisor **Mr. Chiheb Rebai** for his constant support, supervision, contributions, and encouragement not only in this project but also throughout my studying years at Sup'Com.

Finally, I would like to express my gratitude to my school friends and my colleagues at EY **Sachin Shelar** and **Rémi Dendievel** for their help and encouragement that enabled me to carry out this work.

Abstract

This report was prepared as part of the end of a study project within Ernst & Young to obtain the Telecommunications Engineering degree from the Higher School of Communications of Tunis.

The project is about an implementation of a customized Named Entities Extraction solution that is capable of processing unstructured business documents and extracting the key entities from them.

In this context, we have scraped scanned business documents PDF files .Then we processed them in order to convert them into texts, and we managed to annotate the data that we collected utilizing the Active Learning process.

After preparing the training data we leveraged Transfer Learning for creating a named entities recognition model that extracts the main named entities like names, functions, dates, etc. The model was created based on a CamemBERT transformer using mainly the Spacy library which results in an F-score of 78.91%.

The primary aim of this project is to create a tool that, starting from an unstructured Business document, automatically extracts the main information from text by recognizing the named entities mentioned in it.

Keywords : Business Documents Processing , Natural Language Processing, Named Entities Recognition , Active Learning, Transfer Learning

List of Acronyms

NLP Natural Language Processing
NER Named Entity Recognition
TL Transfer Learning
AL Active Learning
AI Artificial Intelligence
CRISP-DM Cross-Industry Standard Process for Data Mining
OCR Optical Character Recognition
ICR Intelligent Character Recognition
ML Machine Learning
NN Neural Network
RNN Recurrent Neural Network
LSTM Long Short-Term Memory
CNN Convolutional Neural Network
HTTP Hypertext Transfer Protocol
BERT Bidirectional Encoder Representations from Transformers
JSON JavaScript Object Notation
AJAX Asynchronous JavaScript and XML
VPN Virtual Private Network
HTML HyperText Markup Language
DOM Document Object Model
SME Subject Matter Expert
XML Extensible Markup Language
MB Moniteur Belge
NLTK Natural Language Toolkit
POS Parts of Speech
QBC Query-by-Committee
LM Language Model
MLM Masked Language Model
NSP Next Sentence Prediction

Contents

General introduction **1**

1 Project Context & Scope Definition **2**
1.1 Introduction . . . 2
1.2 Hosting company presentation . . . 2
1.2.1 Ernst & Young . . . 2
1.2.2 EY Services . . . 2
1.2.3 EY Innovation & Technology . . . 3
1.3 Project presentation . . . 4
1.3.1 Scope . . . 4
1.3.2 Problem description . . . 4
1.3.3 Proposed solution . . . 5
1.4 Project management approach . . . 5
1.4.1 Meetings & team work . . . 6
1.4.2 CRISP-DM methodology description . . . 6
1.4.2.1 Understanding the Business Problem . . . 6
1.4.2.2 Understanding the data . . . 6
1.4.2.3 Data preparation . . . 6
1.4.2.4 Modeling . . . 7
1.4.2.5 Evaluation . . . 7
1.4.2.6 Deployment . . . 7
1.5 Conclusion . . . 8

2 The AI based techniques for entities extraction **9**
2.1 Introduction . . . 9
2.2 Natural Language Processing (NLP) . . . 9
2.2.1 Definition . . . 9
2.2.2 Key Application Areas of NLP . . . 9
2.2.3 Typical text pre-processing tasks in NLP . . . 10
2.2.3.1 Sentence Segmentation . . . 10
2.2.3.2 Word tokenization . . . 10
2.2.3.3 Stemming and lemmatization . . . 10
2.2.3.4 Stopword removal . . . 10
2.2.3.5 Noise removal . . . 11
2.2.3.6 Parts of Speech tagging . . . 11
2.2.3.7 Word embeddings . . . 11
2.2.4 Named Entites Recognition (NER) . . . 12
2.2.4.1 Definition and specifications . . . 12
2.2.4.2 Deep Learning Techniques for NER . . . 13
2.3 Transfer Learning . . . 15
2.3.1 Definition . . . 15

2.3.2 Traditional Learning vs Transfer Learning . . . 15
2.3.3 Transfer learning strategies and techniques . . . 16
2.3.4 Transformers . . . 17
2.3.5 Bidirectional Encoder Representations from Transformers (BERT) . . . 17
2.3.5.1 BERT Pre-training . . . 18
2.3.5.2 BERT Fine-Tuning . . . 18
2.3.5.3 The added values of BERT . . . 19
2.4 Active Learning . . . 19
2.4.1 Definition . . . 19
2.4.2 Active Learning categories . . . 20
2.4.2.1 Stream-based selective sampling . . . 20
2.4.2.2 Pool-based sampling . . . 20
2.4.2.3 Membership query synthesis . . . 21
2.4.3 Active learning querying strategies . . . 21
2.4.3.1 Uncertainty sampling . . . 21
2.4.3.2 Query-by-Committee (QBC) . . . 22
2.4.3.3 Expected Impact . . . 22
2.4.3.4 Density-Weighted Methods . . . 22
2.5 Conclusion . . . 22

3 Data Collection and Preparation Process 23
3.1 Introduction . . . 23
3.2 Data Collection process . . . 23
3.2.1 Required Data . . . 23
3.2.2 Identifying data source . . . 23
3.2.3 Data collection method and tools . . . 24
3.2.3.1 Web scraping . . . 24
3.2.3.2 Scrapy . . . 24
3.2.3.3 Selenium . . . 24
3.2.4 Scrapy VS Selenium . . . 25
3.2.5 Data collection workflows . . . 25
3.2.5.1 Selenium workflow Architechture . . . 25
3.2.5.2 Scrapy workflow Architechture . . . 26
3.3 Data preparation . . . 27
3.3.1 Optical character recognition processing (OCR) . . . 27
3.3.1.1 Definition . . . 27
3.3.1.2 OCR techniques . . . 27
3.3.1.3 OCR processing tool: Tesseract . . . 27
3.3.2 Data annotation . . . 28
3.3.2.1 The different data annotation approaches . . . 29
3.3.2.2 Annotation Tool . . . 30
3.3.2.3 Data labeling guide . . . 30
3.3.2.4 Annotation with Active Learning . . . 32
3.3.3 Data preparation workflow . . . 33
3.4 Conclusion . . . 33

4 Solution realization and evaluation 34
4.1 Introduction . . . 34
4.2 Solution workflows: Build & Production . . . 34
4.3 Setup of the Work Environment . . . 35
4.3.1 Hardware environment . . . 35

4.3.2 Software environment . . . 35
4.3.3 Frameworks and libraries . . . 36
4.4 Evaluation Metrics . . . 37
4.4.1 Beam certainty Score . . . 37
4.4.2 Precision . . . 37
4.4.3 Recall . . . 37
4.4.4 F Score . . . 37
4.5 Modeling . . . 38
4.5.1 SpaCy French Blank Model . . . 38
4.5.2 SpaCy CamemBERT Transformer . . . 39
4.6 Results and interpretations . . . 41
4.6.1 Data Collection result . . . 41
4.6.2 Active learning results and model performance through iterations . . . 41
4.6.3 Named entities Recognition models performance . . . 44
4.7 Graphical user interface . . . 45
4.8 Project effective realization chronogram . . . 47
4.9 Conclusion . . . 47

Conclusion and perspectives **48**

Bibliography Netography **49**

Appendix A **51**

List of Figures

1.1 CRISP-DM project life cycle [1] . . . 7
1.2 Internship Tasks Planning Time Table . . . 8

2.1 NLP pre-processing example . . . 11
2.2 Word2Vec and GloVe word embeddings . . . 12
2.3 Convolutional neural networks architecture in NLP . . . 13
2.4 LSTM Cell Architecture . . . 14
2.5 Traditional machine Learning VS Transfer Learning . . . 15
2.6 The different transfer learning strategies and techniques . . . 16
2.7 The Transformer – Model Architecture[30] . . . 17
2.8 BERT pre-training model and on NER fine-tuning Architecture . . . 18
2.9 Stream-based selective sampling process . . . 20
2.10 Pool-based sampling process . . . 20

3.1 Selenium workflow Architechture . . . 26
3.2 Scrapy workflow Architechture . . . 26
3.3 Tesseract OCR engine architecture[17] . . . 28
3.4 Data Labeling approaches[11] . . . 29
3.5 Screenshot of entities labeling using Doccano . . . 31
3.6 Active learning annotation cycle . . . 32
3.7 Data preparation pipeline . . . 33

4.1 Solution building workflow . . . 34
4.2 The end-to-end solution workflow . . . 35
4.3 SpaCy NER Architecture . . . 38
4.4 SpaCy Blank NER Model configuration . . . 39
4.5 SpaCy NER Architecture with transformer . . . 39
4.6 CamemBERT SpaCy configuration and characteristics . . . 40
4.7 The density distribution of certainty score in each iteration . . . 42
4.8 Percentage of docs with good prediction certainty evolution through iterations . . . 43
4.9 Application Main page . . . 45
4.10 The application use methods . . . 46
4.11 Internship Tasks Realization Time Table . . . 47

List of Tables

4.1 Data Collection Result 41
4.2 Files count in each Active Learning iteration 41
4.3 The Entity Counts of the labled data created with Active Learning 43
4.4 SpaCy Blank Model & CamemBERT transfomer performance evaluation 44

Introduction

A large number of bylaw documents are conducted and published each year. Banks and insurance companies must process their client's company bylaws, either for new client onboarding or to keep existing client data (e.g. legal representatives, functions, etc.) up to date.

Traditionally, bylaws documents are processed manually. Employees have to read company bylaws publications end to end, manually extract the necessary information and encode them within the company's back-office applications. This approach consumes a lot of time and energy and is inefficient because the documents are unstructured, complex, and do not always follow a clear template, and with the exponential increase in the number of those documents, it is becoming increasingly difficult for employees to keep up with all of the updated information.

Recently, many systems based on Natural Language Processing techniques have been published to bring new approaches to text processing in a manner that reveals specific information, such as entities, the relationships and patterns buried in the text, which is normally imperceptible to readers. Entity recognition is one of the most useful approaches, which consist of the automatic identification of named entities in texts, typically with their assignment to a set of semantic categories.The established classes are typically person, organization etc.

The goal of this internship is to leverage Natural Language Processing, Transfer Learning, and Active Learning to create an information extraction solution that extracts the main named entities in a given document, which will make it easier and quicker for employees to perform their task.

The first chapter contains a high-level overview of the project, beginning with presenting the hosting company, its main highlights, then giving the project description and the working approach. Throughout the second chapter, we define the AI-based techniques for entities extraction including Natural Language Processing, Transfer Learning, and Active Learning, outlining the various scenarios and approaches that we can employ.
In the third chapter, we focus on the data specifying the collection techniques, the preprocessing steps, and the Active Learning annotation process to finally get the required data ready.
After that, the solution realization and evaluation is the main focus of the fourth chapter. We also walk through the various technologies used, the transfer learning modeling, and the performance evaluation and display the web application graphical interface of the final product.

And finally, we will wrap up our project with a conclusion and the perspectives for future work.

Chapter 1

Project Context & Scope Definition

1.1 Introduction

In this chapter, we present an overview of the general context of the project. We begin with an introduction of the host company and its services. Then, in the second part we focus on the project idea, which problem it solves, and its goal. Finally, we present the project management approach that enabled us to achieve the work successfully.

1.2 Hosting company presentation

1.2.1 Ernst & Young

Ernst & Young or simply EY is founded in 1989 [3]. It is one of the largest professional services networks in the world and one of the Big Four accounting firms with more than 300 000 people in more than 150 countries.

EY is one of the world's largest providers of assurance, tax, transaction, and consulting services. Thier insights and high-quality services contribute to the growth of trust and confidence in capital markets and economies throughout the world.

Today, Artificial Intelligence is causing major disruption in many sectors, and that is why EY is making major investments in this field to build cutting-edge AI-based tools to provide better services to its clients.

1.2.2 EY Services

EY has transformed its business model and broadened the range of services it provides over the course of its activities. EY's business model has evolved significantly over the last decade in order to include a broader range of services.

This is primarily due to increased competition in the traditional professional services industry as well as competition in emerging sectors such as investment banking and strategic consulting. The following are the company's four main service lines:

— **Assurance:** It consists of Financial Audit, Financial Accounting Advisory Services, and Forensic & Integrity Services. Through this service EY helps provide an objective and independent examination of the financial statements that are published to investors and other stakeholders.

— **Tax:** EY helps build proactive, truly integrated direct and indirect tax strategies to recognize opportunities in business changes and build sustainable growth through Transfer Pricing, International Tax Services, Business Tax Compliance, People Advisory, Global Trade, Indirect Tax, Tax Accounting & Risk Advisory Services, Tax Technology and Transformation and Transaction Tax.

— **Consulting:** It consists of helping transform businesses through the power of people, technology and innovation and it divided into two sub-service lines – Business Consulting and Technology Consulting.

— **Strategy and Transactions or SaT:** Through this service EY helps clients reimagine their ecosystem, reshape their portfolios and reinvent themselves for a better future. It deals with company's capital transformation, including Business Valuation and Economics, Due Diligence, Real Estate Advisory, Restructuring (financial and operational), Corporate Finance Strategy.

1.2.3 EY Innovation & Technology

EY Global Innovation teams work across the firm to help discover, share, and develop the brightest ideas, as well as research and explore emerging technologies, particularly in the AI industry.
They are developing a strategy that focuses on establishing an AI-enabled organization by bringing together the capabilities of multidisciplinary teams.

In reality, EY's approach focuses on AI-led transformation rather than just automation, assisting clients in unlocking the value of data. As a result, in 2020 – for the third consecutive year – EY was recognized as a Leader for data and analytics services by Gartner.

The EY team has extensive global and local expertise, backed up by global partnerships and market studies, and has established a complete set of tools to make the AI journey faster. To mention some of those tools there is:

— **Document Intelligence Suite** that automates handling of unstructured data within documents using AI.
— **EY Smart Information Retrieval Engine (SIRE)** a tool that extracts information from documents and understand context and meaning etc.
— **Readiness and maturity assessment framework** a framework and related questionnaires and checks to guide AI readiness assessment (existing capabilities and gaps).
— **AI and analytics workbench** it consist of a secure platform for analysis of large, complex datasets for the purposes of one-off data analysis and rapid development of PoC or Prototypes.
— **Cognitive Investigator** it is an innovative technology platform utilizing AI & ML, advanced analytics and RPA to support an organization's financial crime investigations process.
— **Trusted AI framework** it consist of framework for AI risk assessment and management performance, bias management, resilience, root causes, transparency.

— **Data Quality Machine Learning (DQML)** it's a tool generating reports, ensuring alignment with data quality regulations, identifying potential new rules through detecting issues.

— **AI Technology Radar** it provides an overview of AI tools (based on application, type of technology, vendor, maturity etc.).

— **AI use case repository** it consist of a use case repository covering multiple industries and value chain parts to facilitate maturity assessment and faster identification of value creation opportunities.

Indeed EY continues to be one of the fastest-growing providers in consulting services with Data and AI Technology as one of the top pillars of EY firm-wide strategy with more than 27,000 Data Analytics, AI, and Intelligent Automation practitioners and around 9 Artificial Intelligence Labs.

1.3 Project presentation

1.3.1 Scope

This project was realized in the context of a graduation internship at the Higher School of Communication of Tunis, and it is one of the requirements for graduation and getting the national engineering diploma. It specifies implementing the student in an internship within a professional environment from 4-6 months.
This project was done at EY from 01/03/2021 to 30/06/2021.

1.3.2 Problem description

Extracting significant information from business documents like invoices, contracts, etc, represents a real challenge for all large companies.
Traditionally, banks and insurance companies have had to manually process their client's company bylaws, either for the on boarding of new clients or to keep up with the data of existing clients. This approach is time-consuming and inefficient given that the document does not have a specific structure or template.

That is why companies have a real need to rethink the traditional approach of document processing and automation and leverage the existing technologies in combination with AI and machine learning.
Furthermore, deep learning has radically changed sequence modeling, which leads to end-to-end trained systems with high-performing intelligent document processing that improves the process automation for a wide variety of use cases, streamlining operations and boosting workforce productivity while minimizing cost and business disruption.

The aim of this project is to leverage NLP, Transfer Learning, and Active Learning in order to build an information extraction solution which should help the employees and to make thier job easier.

1.3.3 Proposed solution

Our prposed solution consists of implementing an end-to-end solution that is able to extract a set of attributes from a publicly published company's bylaws. The solution includes:

— A web scraping solution for gathering the company's bylaws files. The solution have to be capable of collecting around 400 document written in french.

— A document prepossessing solution that uses an Optical Character Recognition technique to transcribe scanned documents into texts. In order to provide clear text, the solution must be capable of processing unstructured documents and of dealing with poor scanning quality (blurry text, stamps, noise, etc.).

— Data annotation of the processed documents using an Active Learning approach. The labels used in the annotation should correspond to the information that the employee wants to extract and should be simple and easy to identify. At least 300 documents should be annotated to have, in the next step, a good model prediction.

— Building a Named Entities Recognition model by fine-tuning and customizing a pretrained one and evaluate it using Precision, Recall and F score metrics. Moreover the model should provide a performance good enough to have a relevant entities recognition according to previous similar works having a model with more than 75% F score would give a good NER result [31] [15].

— Combining the document preprocessing module and the model to build an end-to-end final solution that takes as an input a new document and gives as an output the extracted entities from it.

Finally, the solution should allow the user to automatically extract the main named entities from scanned documents.

1.4 Project management approach

The work approach plays a major role in project realization and success. That's why we have to choose a methodology that responds to the project requirements and makes the work process more efficient.

Considering the nature of our project which is based on artificial intelligence and involves business understanding, data preparation, modeling, evaluation, and finally the deployment we choose the agile approach Cross-industry standard process for data mining(CRISP-DM) that suits very well the requirements of the project.

1.4.1 Meetings & team work

We worked on this project as members of the Data Science & AI EY Team and in our working process we had two types of meetings :

— **Daily Meeting:**
Duration: 30 minutes
This meeting is held daily between the project team members.
The daily meeting's main advantage is that it is fast and efficient . The aim of this Meeting is that each member must discuss the following point:
What we have done yesterday? What we will do today? What are the blockers we encountered?

— **Biweekly meeting**
Duration: 1 hour
This meeting is held every two weeks between the project team members, the supervisor and other Data Science & AI teams members .
In this meeting each team present the work that they achieved and discuss what to improve in the previous work and what's the next step.
It's comparable to Sprint Planning, and it's about ensuring that the team has plenty of the right things to complete , that they can stick to completing them and tack the feedback of other team members .

1.4.2 CRISP-DM methodology description

The CRISP-DM method was originally developed by IBM in the 1960s to run Data mining projects.
The method is divided into 6 stages, from understanding the business challenges to deployment as Figure 1.1 shows:

1.4.2.1 Understanding the Business Problem

The first step is to understand the business elements and problems that artificial intelligence aims to solve or improve.
In our project, we focused on understanding the problems with the traditional bylaws document processing, as well as how we may apply AI techniques to automate that task.

1.4.2.2 Understanding the data

The aim of this step is to precisely specify the data that we needed, and we analyzed the quality of the existing data to identify the main information that must be retrieved in order to offer the required solution.

1.4.2.3 Data preparation

This step includes activities related to the construction of a set of the required data to be analyzed. It, therefore, includes the documents file processing and format transformation as well as text labeling. In order to have data prepared and compatible with the algorithms to be used.

1.4.2.4 Modeling

Modeling includes the choice, the parameterization and the testing of various algorithms as well as their sequence, which constitutes a model.

1.4.2.5 Evaluation

The goal is to validate the models and algorithms used, as well as the knowledge acquired, using various metrics such as precision, recall, and F score to ensure that they meet the objectives. It also helps, through analyzing the results, determining what improvements can be made to achieve better results the next time.

1.4.2.6 Deployment

This is the last step in the process. This includes putting the resulting model into production for the end user. Its objective is to place the knowledge acquired through modeling in an appropriate form and to integrate it into the decision-making process.
Therefore, we deployed the solution that we created as a simple application that can be used by anyone.

Figure 1.1 shows the general CRISP-DM project life cycle starting from Business understating to the deployment.

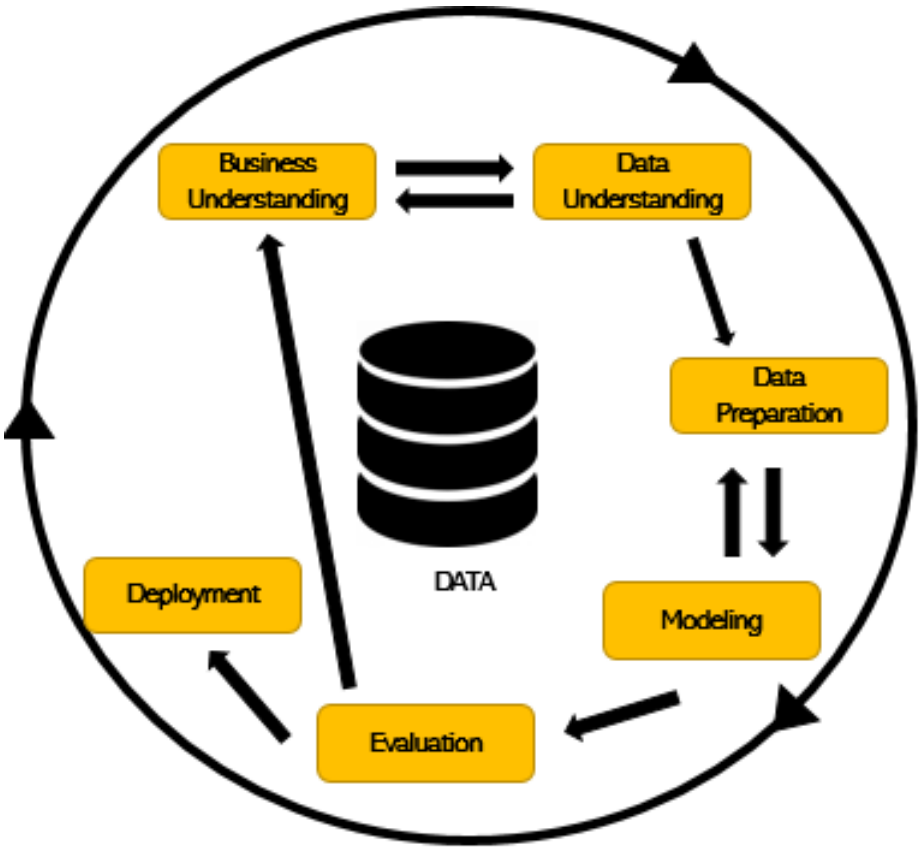


Figure 1.1 – CRISP-DM project life cycle [1]

Figure 1.2 also presents in details the time planning of every task and subtask in our project
.

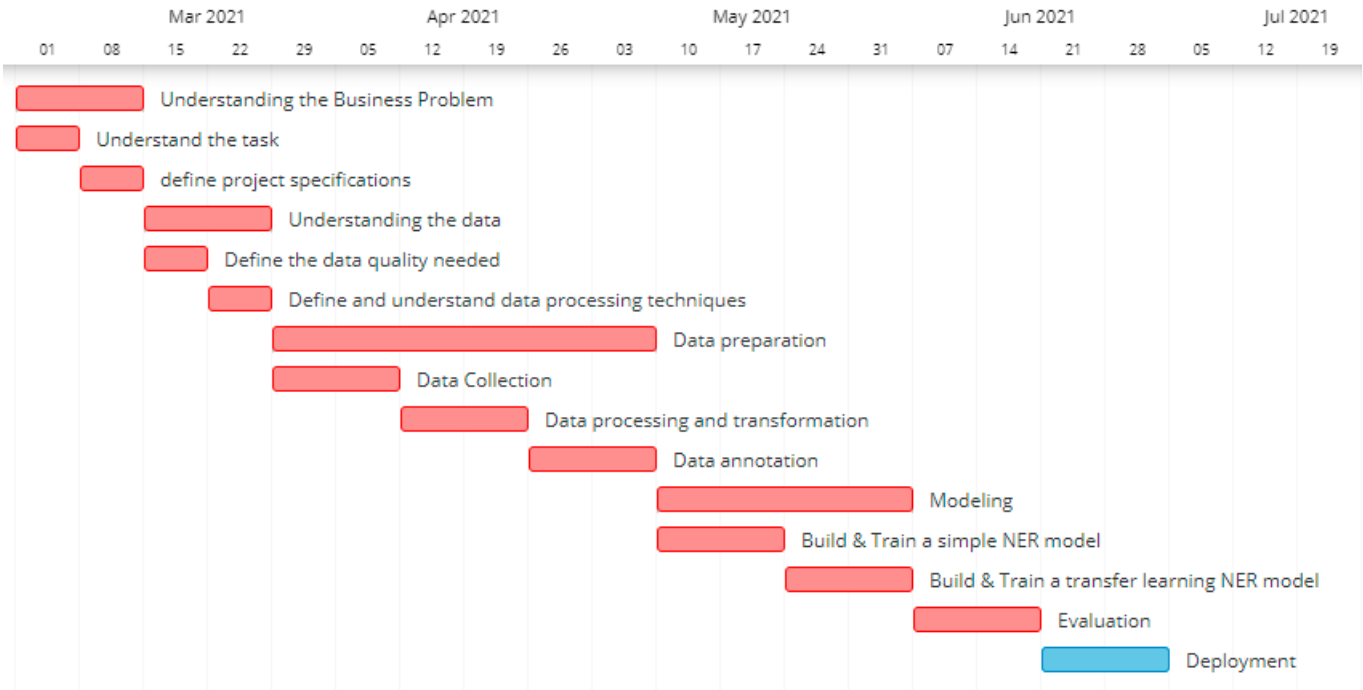


Figure 1.2 – Internship Tasks Planning Time Table

1.5 Conclusion

In this chapter we highlighted the context of our project presenting the host company and its activities, describing the problem and proposing a solution .
The chapter also presented the specification and the required work to realize the proposed solution in addition to the project management approach which is CRISP-DM detailing the meeting planning , the project life cycle and the time tasks planning. In the next chapter we explain the backgroud theory of the AI-based techniques that we used for entites extraction.

Chapter 2

The AI based techniques for entities extraction

2.1 Introduction

In this chapter, we present the state of the art of the AI based techniques for entites extraction which includes Natural Language Processing as a technique to process the text data to extract the main information , the Active Learning approach that helps annotating data in a fast and effective way and the Transfer Learning technique that help to build an effective model that gives a good result using few amount of data and limited ressources.

2.2 Natural Language Processing (NLP)

2.2.1 Definition

Natural language processing (NLP) is a subfield of linguistics, computer science, and artificial intelligence that focuses on studying the interactions between computers and human language, especially how to create an algorithm that can process and analyze large amounts of natural language data.
This gives the computers the ability to understand the contents of documents as well as the inner contextual nuances of the language. The NLP technology is able to accurately extract information and meanings contained in documents in addition to categorizing and organizing the documents themselves.[20]

2.2.2 Key Application Areas of NLP

— **Search :** It consists of identifying certain elements in the text. It gives the ability of searching based on keywords in a document, running a contextual search for synonyms, detecting the existence of misspelled words or similar entities, etc.

— **Machine translation :** This commonly implicates translating one natural language into another, keeping the meaning, and generating fluent text as an output.

— **Summarization :** NLP programs can be used to create a version of articles and long texts that is shorter and contain the main information with main points and key ideas included.

— **Named-Entity Recognition :** NER is an entity identification, extraction, and categorization. It implicates extracting names of different entities like people, locations and putting them under specific classes and since in our project we are going to implement a NER solution we will give more details about it in the next parts.

— **Text classification :** Grouping, or text clustering, is realized with the text tags. The NLP algorithm is trained to classify documents based on specific features: subject, document

type, time etc.

— **Sentiment analysis** It's a category of text classification in which the NLP model decides the text's positive, negative, or neutral connotation. Use cases involve analyzing client's feedback, conducting market research, etc, through tweets analysis, posts and reviews.

— **Answering queries :** An automated question answering system uses a set of NLP methods to analyze unstructured data from articles to social media newsfeed or medical records by extracting the needed information elements, analyzing it, and using the relevant part to answer the question.

2.2.3 Typical text pre-processing tasks in NLP

Most NLP solutions include a text pre-processing pipeline and tasks that should be processed in order to effectively transform the text into a format that can be consumed by models and other tasks.

2.2.3.1 Sentence Segmentation

It involves breaking the text into separate sentences. Coding a sentence segmentation model may be as easy as separating sentences anytime a punctuation mark appears.
However, newer NLP pipelines frequently employ more advanced algorithms that operate even when a document is not properly structured.

2.2.3.2 Word tokenization

Tokenization is the process of dividing a piece of text into smaller components known as tokens. Tokens can be words, letters, or subwords in this context.
As a result, tokenization may be generally categorized into three types: word tokenization, character tokenization, and subword tokenization. The basic reason for tokenizing text data is that the most common techniques of processing raw text happen at the token level.

2.2.3.3 Stemming and lemmatization

Text Lemmatization and Stemming in NLP used for grammatical reasons, documents can contain different forms of a word such as drive, drives, driving. Also, sometimes we have related words with a similar meaning, such as nation, national, nationality.
Stemming is a rule-based method of removing the tails of words. While text lemmatization is a more specialized process that uses dictionaries and morphological analysis of words, it is commonly done by having a look-up table of the lemma forms of words depending on their part of speech and potentially some custom rules to handle words that have never being seen before.

2.2.3.4 Stopword removal

Since a lot of state-of-the-art NLP techniques work on the lexical level using statistic approaches, this step removes the terms that are widespread and frequent in any kind of text, and hence are not informative about the specific text content.
Stop words are usually identified just by checking a hardcoded list of known stop words. But there's no standard list of stop words that is appropriate for all applications. The list of words to ignore can vary depending on the application.

2.2.3.5 Noise removal

Noise removal is the act of cleaning the data to get rid of errors and uninteresting features like spelling correction, dealing with capitalization , accented or special characters and word boundaries. And all this to improve the NLP model's performance.

Figure 2.1 shows an example the NLP reprocessing techniques on a sentences.

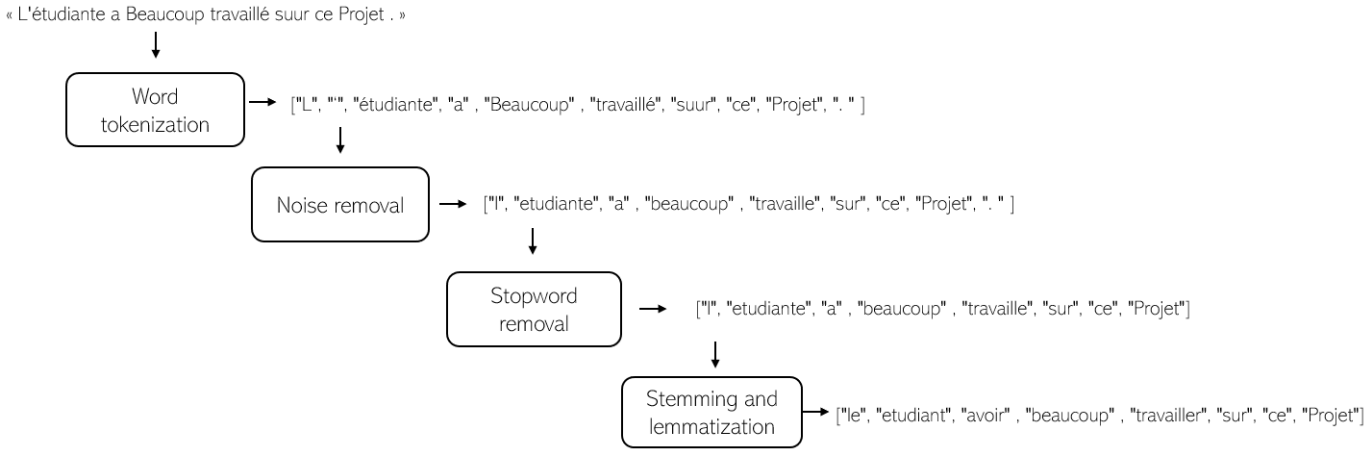


Figure 2.1 – NLP pre-processing example

2.2.3.6 Parts of Speech tagging

A POS tag (or part-of-speech tag) is a special label assigned to each token (word) in a text corpus to indicate the part of speech and often also other grammatical categories such as tense, number (plural/singular), case etc. POS tags are used in corpus searches and in text analysis tools and algorithms.
The model is originally trained by giving it millions of phrases with each word's part of speech already marked and letting it learn to reproduce that behavior.
So, it is completely dependent on statistics, which means it does not understand what words mean in the same way that people do. It only understands how to estimate a segment of speech based on previous experiences with comparable phrases and words.

2.2.3.7 Word embeddings

This method involves converting each word, which is now an integer value, into a vector of float values. These vectors are known as dense vectors, and they are used to describe a word's meaning in a higher dimension.
Embedding methods are small neural networks in and of themselves, and the most well-known ones are GloVe (Global Vector) and Word2Vec.

Word2Vec [18] attempts to anticipate which word will appear given the context of the text. Given a statement such as "My favorite pet is...", it most likely predict that "dog" and "cat" have comparable probability.
As a result, Word2Vec implies that the words "dog" and "cat" are semantically related in this context and will have extremely close embeddings on at least one level.

GloVe[24] differs from Word2vec in that it is a learning-not-follow model. It is based on the construction of a global co-occurrence matrix of terms where each element is the frequency of a word that appears in the context of the word.
Thanks to a sliding window, Glove takes into account all information carried by the corpus and not only from the word window.
Once the co-occurrence matrix has been established, the model learns the representations of terms so that their dot product is equivalent to the logarithm of the probability of co-appearance of the associated terms.
Figure 2.2 illustrates the Word2Vec and GloVe word embedding word representations.

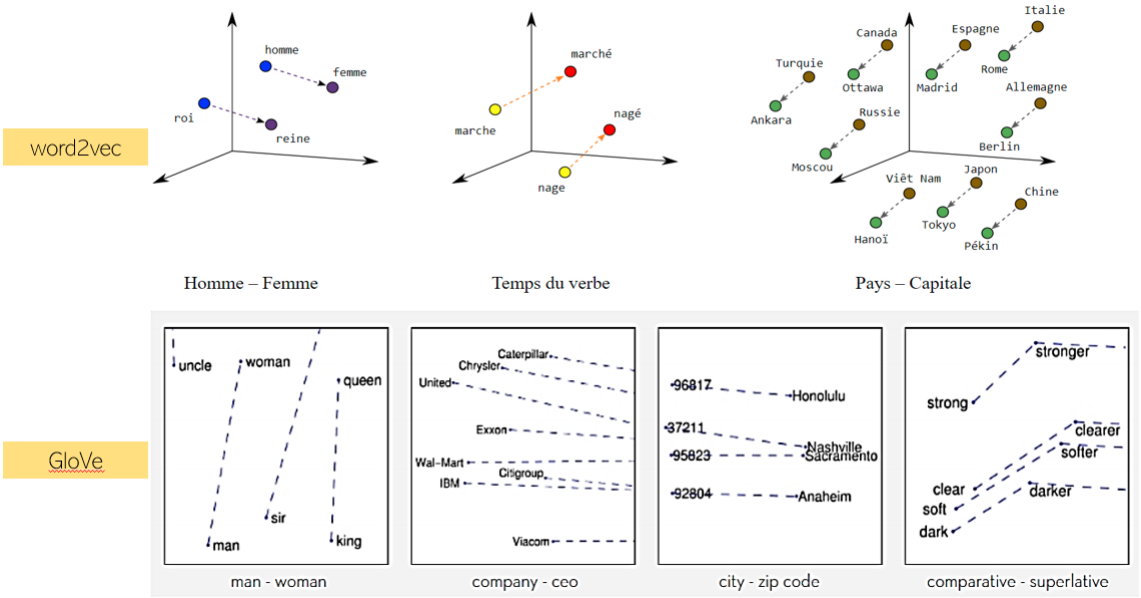


Figure 2.2 – Word2Vec and GloVe word embeddings

2.2.4 Named Entites Recognition (NER)

2.2.4.1 Definition and specifications

Named entity recognition (NER) was first introduced during The Sixth Message Understanding Conference (MUC-6) in 1995.
It aims to identify occurrences of general concepts, including names of people, places and organizations or quantities such as currencies, dates and percentages in the text. The NER can also be thought of as a classification task that assigns each named entity to its correct class, concept or entity[19].
NER techniques are generally adapted based on the following factors:

— **Entity types** are also referred to as categories or concepts of entities. The simpler NER techniques are used to identify big concepts grains such as person, place and organization. It is also common to identify fine-grained entities such as people of various occupations, such as doctors and teachers, or specific people.

— **The Areas** such as finance, medicine or information technology and types of text such as journalistic, scientific or informal must be taken into account when designing a NER algorithm. The adaptation of NER techniques from one domain to another has bad implications for accuracy.

— **The Language** is another important factor to consider. English has so far received the greatest attention in the NER field compared to other languages, however, massive amounts of texts in foreign languages are published online. French is not as popular as English and many more difficult due to the lack of annotated data and knowledge bases.

2.2.4.2 Deep Learning Techniques for NER

In order to improve the performance on NLP tasks and especially NER , researchers tend to use the deep learning methods specially the Convolutional Neural Network and Long short-term memory.

— **Convolutional Neural Network (CNN)**:
Convolutional neural networks CNNs, [13] which were first employed in image processing, are now widely employed in a variety of NLP tasks. We can use CNN to successfully extract local information from tokens instead of image pixels.
The input to most NLP tasks are sentences or documents represented as a matrix. Each row of the matrix corresponds to a single token, which is usually a word but might be a character.

In fact, each row is a vector that represents a word. These vectors are often word embeddings (low-dimensional representations) like word2vec or GloVe, but they might also be one-shot vectors that index the word into a vocabulary.
For example a 10 word sentence we use a 100-dimensional embedding we would have a 10×100 matrix as our input. That's considered an "image" in computer vision case.

The varied size filters are then applied to the embedding matrix to capture essential aspects of nearby inputs. As a result, the "width" of our filters is frequently equal to the width of the input matrix. The height or region size may vary, but often, sliding windows over 2-5 words at a time are used.

In the final CNN process, a max-pooling operation is done to extract a single feature for all feature maps. The output characteristics are then concatenated to represent each word, preserving the local information.
The Figure 2.3 shows the typical CNN architecture in NLP.

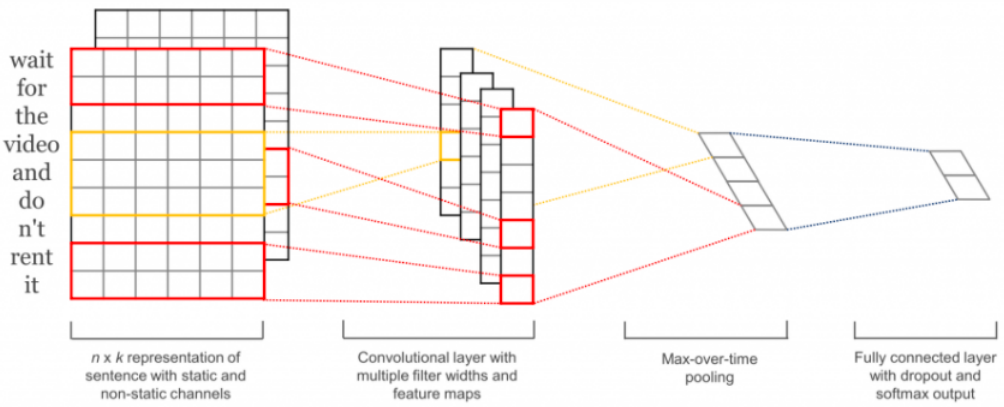


Figure 2.3 – Convolutional neural networks architecture in NLP

Following researches showed that CNN is effective in the feature engineering. However, the CNN pays more attention to learn the spatial features, while the text is mainly represented by temporal features. So to improve performance researchers tried to use Long Short-Term Memory network (LSTM).

— **Long short-term memory (LSTM)**:
LSTM [29] is an upgraded form of RNN (Recurrent Neural Network) that can learn long-term relationships without neglecting short-term dependencies, therefore eliminating the vanishing gradient problem.
This is accomplished by creating a more sophisticated structure within the LSTM cell known as "Gates."
We input this neural network our sentences, which is going to be a set of dense vectors following the word embedding procedure. Our network is made up of LSTM cells.

A typical LSTM unit is composed of a cell, an input gate w_i , an output gate w_o and a forget gate w_f .
The cell stores values for arbitrary time intervals, and the three gates control the flow of information into and out of the cell.
Figure 2.4 shows the LSTM cell architecture.
The computations within the LSTM cell are expressed by the following formulas:

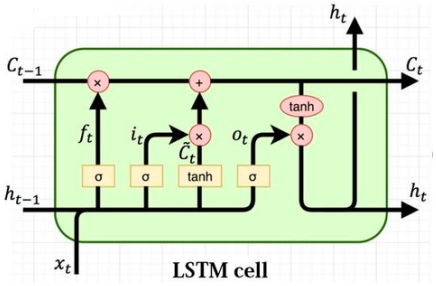


Figure 2.4 – LSTM Cell Architecture

$$i_t = \sigma(w_i[h_{t-1}, x_t] + b_i) \quad (2.1)$$
$$f_t = \sigma(w_f[h_{t-1}, x_t] + b_f) \quad (2.2)$$
$$o_t = \sigma(w_o[h_{t-1}, x_t] + b_o) \quad (2.3)$$
$$c\tilde{}_t = tanh(w_c[h_{t-1}, x_t] + b_c) \quad (2.4)$$
$$c_t = f_t \times c_{t-1} + i_t \times c\tilde{}_t \quad (2.5)$$
$$h_t = o_t \times tanh(c_t) \quad (2.6)$$
$$\sigma = \frac{1}{(1+e^{-x})} \quad (2.7)$$

(2.1): The input gate, w_i denotes the weight of this gate, h_{t-1} the previous cell output, x_t the input, and b_i is the bias at input.
(2.2): The forget gate, w_f denotes the weight of this gate, and b_f is the bias at the forget gate.
(2.3): The output gate, w_o denotes the weight of the output gate, and b_o is the bias at the output gate.
(2.4): Candidate for cell state at timestamp(t), w_c is the weight at this gate and b_c is the bias. This output is used to compute this cell's state.
(2.5): Cell state of the current timestamp, f_t represents the output of the forget gate. c_{t-1} is the previous cell state, and it is the output of the input gate.

(2.6): Input at current timestamp,o_t denotes the output of the output gate, and c_t the output of the cell state.
(2.7): Sigmoid function, this function is used to squish values between 0 and 1.

But LSTM has some drow backs which are that it's slow to train because words are passed and generated sequentially through the neural network in addition two the LSTM is not truly bidirectional so the context is slightly lost.
So researchers tend to use transfer learning techniques and develop transformers that some of the LSTM concern in order to get a higher performance, rapidly and using a few amounts of data.

2.3 Transfer Learning

In this section, we take a deeper look at the concept of transfer learning, its benefits and how it can be used to boost the prediction results.

2.3.1 Definition

Transfer learning is an approach for adapting and applying acquired knowledge to another task related to the task at hand.
In practice, very few individuals build a full NN from scratch since a sufficiently large dataset is uncommon and, if that's the case, the hardware isn't always accessible to train on massive amounts of data. As a result, it is usual practice to run a NN on a very big dataset.
Consequently, it is usual practice to run a NN on a very big dataset, preserve its information (weights, features, etc.), and then utilize it as an initialization or as a fixed feature extractor to jump-start the problem at hand.

2.3.2 Traditional Learning vs Transfer Learning

Unlike traditional learning, which is typically isolated and task-specific, transfer learning knowledge can be moved from one model to another. The knowledge gained from a massive amount of data is then used to solve problems with limited and different datasets.

The Figure 2.5 shows the differences in learning processes between traditional and transfer learning strategies. As we can see, traditional machine learning approaches attempt to learn each task from the scratch, but transfer learning approaches attempt to transfer the gained knowledge from prior tasks to a new work when the latter contains fewer high-quality and different training data.[23]

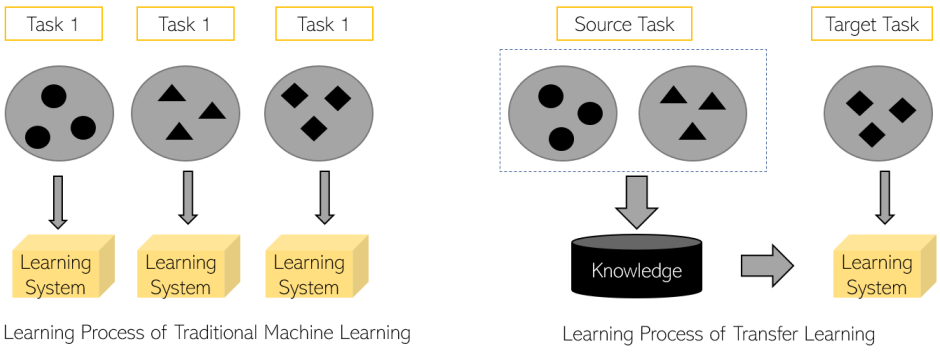


Figure 2.5 – Traditional machine Learning VS Transfer Learning

2.3.3 Transfer learning strategies and techniques

— **Inductive Transfer learning:** [23] The source and target domains are the same in this case, but the source and target tasks are not. The algorithms attempt to use the source domain's inductive biases to help better the target job.
Depending on whether or not the source domain has labeled data, this may be further subdivided into two subcategories, similar to multitask learning and self-taught learning.

— **Transductive Transfer Learning:** There are similarities between the source and target tasks in this instance, but the related domains are distinct.
In this configuration, the source domain has a large amount of labeled data, but the target domain has none. This may be further subdivided into subcategories, which refer to situations in which either the feature spaces or the marginal probabilities differ.

— **Unsupervised Transfer Learning:** This configuration is similar to inductive transfer in that it focuses on unsupervised tasks in the target domain. Although the source and destination domains are identical, the tasks are not. Labeled data is not accessible in either domain in this case.

The different transfer learning strategies and techniques are summarized in Figure 2.6.

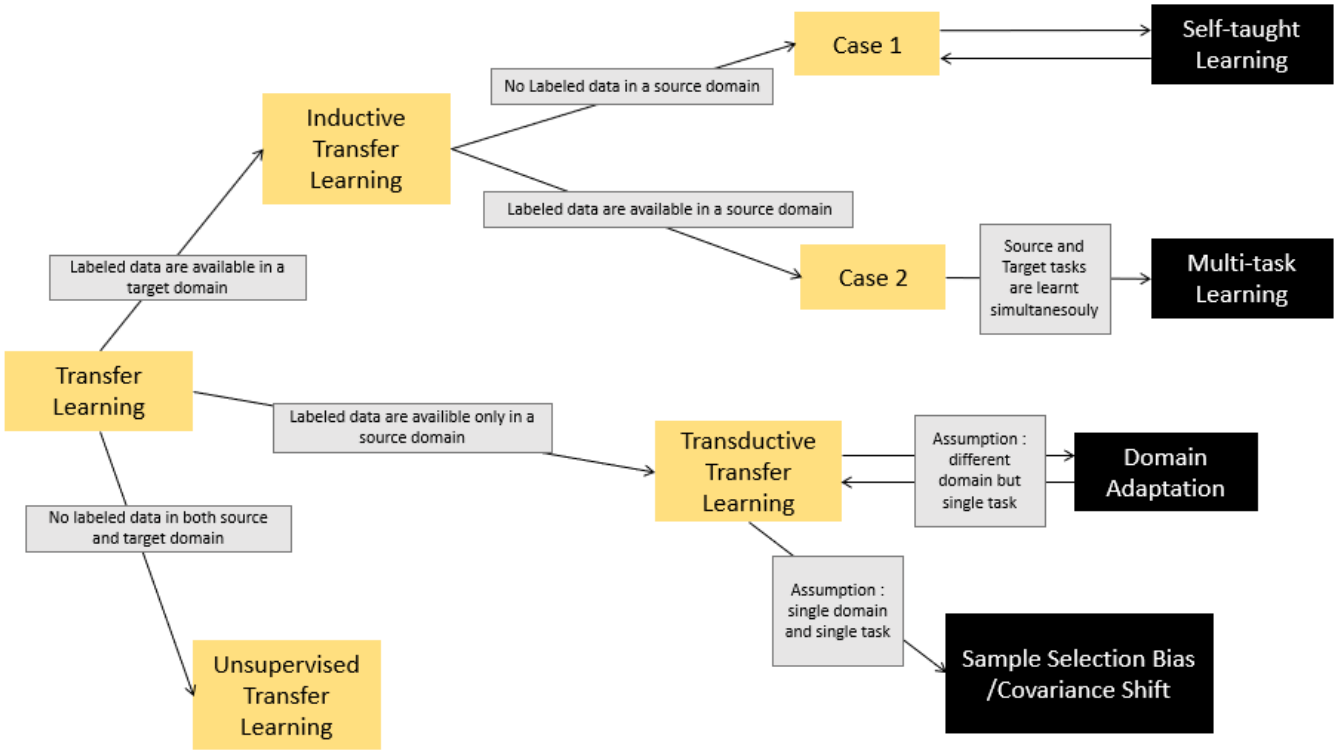


Figure 2.6 – The different transfer learning strategies and techniques

2.3.4 Transformers

The Transformer in NLP is a novel architecture that tries to address sequence-to-sequence problems while also dealing with long-range dependencies. The Transformer was proposed in the paper "Attention Is All You Need" . The concept is to thoroughly manage the dependencies between input and output using attention and recurrence.[30]

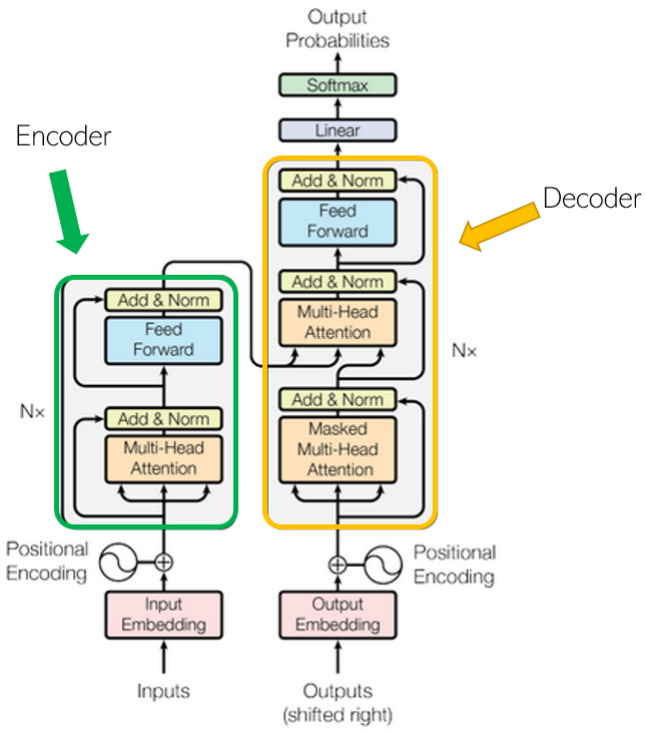


Figure 2.7 – The Transformer – Model Architecture[30]

The Figure 2.7 is an excellent representation of the Transformer's architecture. Focusing on The encoder block which is composed of one layer of Multi-Head Attention followed by another layer of Feed Forward Neural Network. While on the other side, the decoder features an additional Masked Multi-Head Attention.
The encoder takes the input text and transforms each word into embeddings simultaneously. Those embeddings are vectors that encapsulate the meaning of the word.
The decoder takes these embeddings from the encoder and the previously generated words (output shifted right) and then uses them to generate the output.

2.3.5 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) [16] is a Transformer-based machine learning methodology created by Google for NLP pre-training. Jacob Devlin and his Google colleagues designed and released BERT in 2018.

It can be used to extract high-quality language elements from text data, or it can be fine-tuned on a specific job (classification, entity recognition, question answering, etc..) using new data to create state of the art predictions.

There is two model size in BERT:
BERT base which made with 12 Encoders, 12 bidirectional self-attention heads.
BERT large which made with 24 Encoders with 16 bidirectional self-attention heads.
BERT is done in two phases the first phase is pretraining BERT to understand the language and the second phase is finetuning BERT to learn a specific task. Those phases are trained simultaneously. The Figure 2.8 shows a visualization of the neural network architecture of those two phases.
The explanation of each component in the figure is detailed in the next paragraphs.

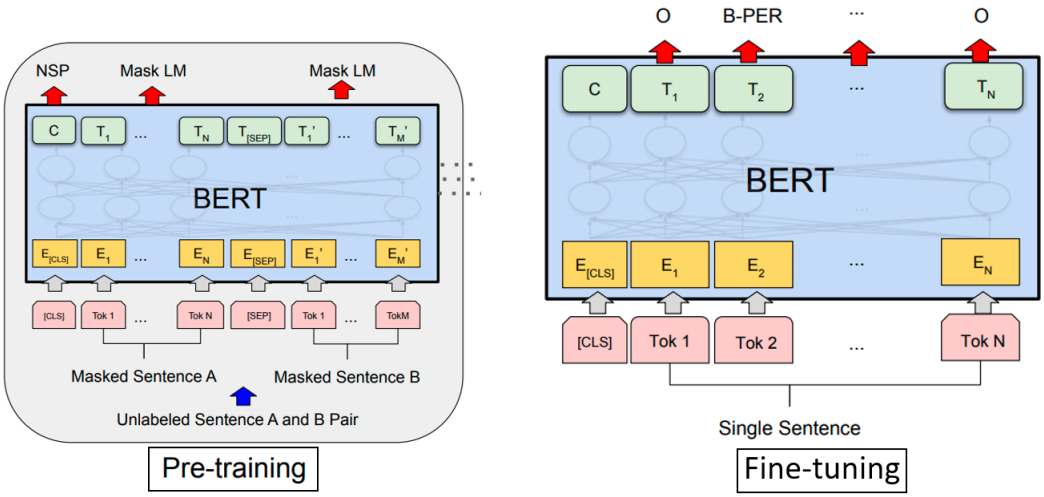


Figure 2.8 – BERT pre-training model and on NER fine-tuning Architecture

2.3.5.1 BERT Pre-training

BERT is pretrained using two unsupervized tasks [16]:

task 1: Masked Language Model To train a deep bidirectional representation, we simply mask a random proportion of the input tokens and then predict those masked tokens. This is referred to as a "masked LM" operation (MLM). As in a regular LM, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary.

task 2: Next Sentence Prediction (NSP) To train a model that understands the relationships between sentences, they pre-train for a binarized next sentence prediction task that can be reasonably generated from any monolingual corpus. Particularly, when selecting sentences X and Y for each pretraining example, Y is 50% of the time the real next sentence that succeeds X (labeled as IsNext) and 50% of the time an arbitrary sentence from the corpus (labeled as NotNext).

2.3.5.2 BERT Fine-Tuning

Fine-tuning is simple since the Transformer's self-attention mechanism allows BERT to mimic numerous downstream jobs simply switching out the relevant inputs and outputs. We just plug in the job-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end for each task.

To summarize in pre-training, the input is a set of two sentences with some of the words being masked and each token is a word as shown in the figure Tok_1 .. Tok_N and we convert each of these words into embeddings E_i using pre-trained embeddings provides a good starting point for BERT to work on.
On the output side, C is the binary output for the next sentence prediction so it would output 1 if sentence B follows a sentence A in context and 0 if sentence B doesn't follow sentence A.
Each of the T_i is a word vector that corresponds to the outputs for the Masked model language task. So the number of word vectors that we input is the same as the word vectors that we output.
In the fine-tuning phase, if we want to perform NER task, we would train the model by modifying the input and the output layer. We pass the sentence each word in a token as input. And the output layer we would put the tokens with their labels (B-PER as shown in the finetuning figure)

2.3.5.3 The added values of BERT

— **Quicker Development :** For instance, the pre-trained BERT model weights already encode a significant amount of information about our language. As a result, training our fine-tuned model takes much less time.[22]

— **Less Data :** Furthermore, and probably more importantly, because of the pre-trained weights, we may fine-tune our job on a lot smaller dataset than would be required for a model generated from scratch.

— **Better Results :** Finally, for a wide range of tasks such as classification, named entites recognition , semantic similarity, question answering, and so on, this simple fine-tuning procedure was shown to achieve state-of-the-art results with minimal task-specific adjustments.

2.4 Active Learning

2.4.1 Definition

Active learning (AL) [25] is learning approach in which a learning algorithm can query an expert user (or other sources of information) interactively to label specific data points with the desired outputs which will bring significant improvement of the ML model performance.
One of the growing issues in machine learning is the large amount of unlabeled data, which is extremely time-consuming to label manually. In these cases, the learning algorithms can actively ask labels from the user/teacher. Active learning refers to this type of iterative supervised learning.

The algorithm in active learning selects the subset of examples to be labeled next from a pool of unlabeled data. Its concept is that if a ML algorithm is given the ability to choose the data from which to learn, it may be able to achieve a higher level of accuracy while using a smaller number of training labels.

As a result, active learners can ask questions interactively during the training phase. These requests typically take the form of unlabeled data instances, and they are directed to a human annotator who tags the instance. This incorporates active learning into the human in the loop paradigm, where it has been a huge success.

There are several scenarios in which active learners may pose queries, and several different query strategies have been used to determine which instances are most informative.

2.4.2 Active Learning categories

Active Learning can be implemented through different scenarios. [25]

2.4.2.1 Stream-based selective sampling

In this case, the active learning algorithm determines whether it is worthwhile to query the dataset for the label of a certain unlabeled input. During training, the model is given with a data instance and must decide whether or not to query the label. This method has limitations since there is no guarantee that the data scientist will remain within the budget.

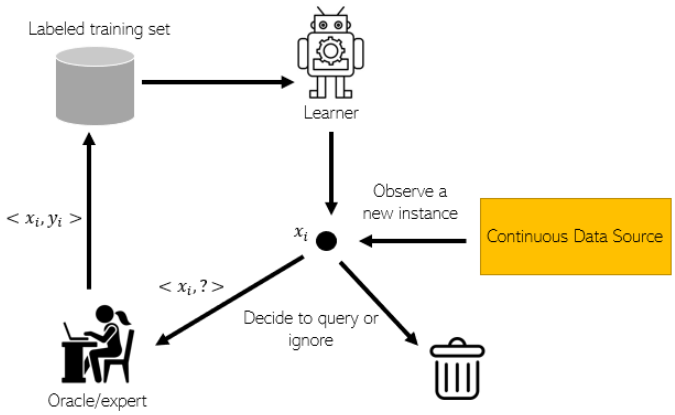


Figure 2.9 – Stream-based selective sampling process

2.4.2.2 Pool-based sampling

Pool-based sampling is the most commonly utilized scenario in active learning. The algorithm in this technique evaluates the entire dataset before selecting the best query or group of question.
The active learner algorithm is often trained on a completely labeled sequence of data at the start, which is then used to choose which instance would be most helpful to include in the training set for the following active learning loop. The downside of this strategy is that it need some memory.[25]

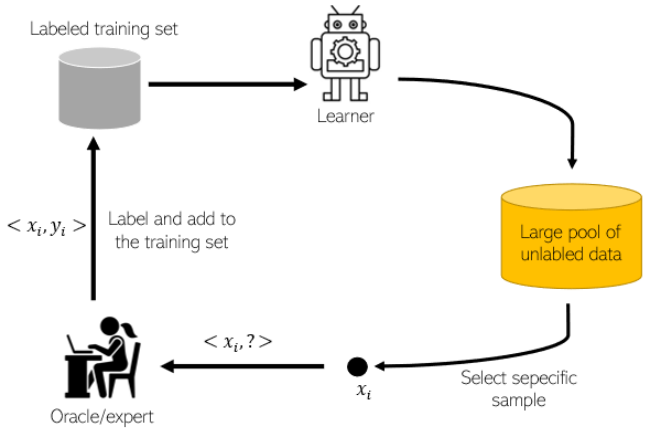


Figure 2.10 – Pool-based sampling process

2.4.2.3 Membership query synthesis

Because it requires the fabrication of synthetic data, this approach cannot be applied in all instances. In this case, the active learner can generate its own labeling instances.
This strategy is appropriate for issues where producing a data instance is simple, and it overcomes the problem of cold start when producing a data instance is simple.

2.4.3 Active learning querying strategies

A querying strategy is the method used to select which data sample to label next. Those that are most widely used and researched are listed below[25]:

2.4.3.1 Uncertainty sampling

In earlier quering techniques, the model is trained on a limited quantity of labeled data and then applied to the rest of the unlabeled dataset.
The algorithm determines which instance to label during the next active learning loop based on the information obtained in the previous phase.

— **Least confidence(LC)** is a popular method in which the algorithm chooses the instance with the lowest confidence level to be labeled next. Mathematically, it takes the highest probability for each data point's forecast and arranges it in ascending order from smaller to larger.
The true term for prioritizing with the least amount of certainty would be:

$$s_{LC} = argmax_x(1 - P(\hat{y}|x)) \ (2.8)$$
$$\hat{y} = argmax_y P(y|x) \ (2.9)$$

— **Margin sampling**: Another sampling strategy in which the model chooses cases where the gap between the two most probable labels is small, indicating that the classifier is having difficulty distinguishing between those two most probable classes.

In that way, the data points with the lowest margin sampling score would be labeled first; these are the data points about which the model is least certain between the most probable and next-to-most probable classes.
The formal mathematical formulation for prioritization is:

$$s_{MS} = argmin_x(P(\hat{y}_{max}|x) - P(\hat{y}_{max-1}|x)) \ (2.10)$$

— **Entropy-based sampling**: The idea of entropy is used in this technique to assess the confidence of a model. If a model is extremely definite about a class given a data point, it will almost certainly have a high certainty for that class, while all other classes will have a low likelihood.

In this instance, the majority of the probability is given to a certain class, and in the case of high entropy, this means that the model distributes the probability evenly across all classes because it is not known which class that data point belongs to.
Formally, the entropy score prioritisation may be defined as follows:

$$s_E = argmax_x(-\sum_i P(\hat{y}_i|x) log P(\hat{y}_i|x))) \ (2.11)$$

2.4.3.2 Query-by-Committee (QBC)

With this approach, a committee of distinct models is trained on the same labeled training dataset and then used to infer on the remaining data (not labeled). The cases with the greatest degree of error are thought to be the most instructive.

This startegy's basic notion is to reduce the version space, which is the set of hypotheses that are relevant with the actual labeled training data.[25]

2.4.3.3 Expected Impact

Those querying techniques are concerned with the influence of adding additional labels on the overall performance of the model. The predicted model change strategy objective is to find the occurrences that, if the labels for those occurrences were known, would cause the greatest change to the actual model.

The anticipated error minimization technique analyzes how much the model's generalization error is anticipated to be minimized rather than how much the model is likely to change. One disadvantage of the approach is that it is the most computationally costly query framework.

To conclude, these approaches analyze the input space as a whole rather than focusing on specific occurrences like uncertainty sampling or QBC would, making them less likely to choose outliers.[25]

2.4.3.4 Density-Weighted Methods

The preceding frameworks described ways for detecting the most relevant occurrences, which have the highest level of uncertainty.
Nonetheless, another way to look at the problem is to focus on the most useful cases in a dataset, which are not necessarily those with the greatest uncertainty, but rather those that are most indicative of the underlying distribution of the data.

2.5 Conclusion

In this chapter, we explain the AI based techniques for entities extraction. We started with introducing NLP and its application areas, common text preprocessing tasks, and the named entity recognition task mentioning some deep learning techniques used to perform it.

Then, we present the fundamentals of Transfer Learning and the cutting edge model in NLP and TL, and finally, we explain the concept of Active Learning and how it helps speed up the annotation task and the possible implementation scenarios.

In the following chapter, we are going to specify the data collection and preparation and implementation process in order to get the training data that we need.

Chapter 3

Data Collection and Preparation Process

3.1 Introduction

The following chapter presents the data collection process of our project. This aspect of the solution is very important, due to the fact that most of the implemented features are going to revolve around the data we gather.
At first, we detail the method and tool that we used to gather the data. Then, we delve into the details behind the data preparation that we have processed to make our data ready and finally we present the annotation process in which we used an Active Learning approach.

3.2 Data Collection process

We started the project with no data, and it is our mission to specify the data that we require, collect it, and process it.

3.2.1 Required Data

In order to successfully realize the name entity recognition, we have to provide our solution with texts extracted from business documents and since our solution has to be a french NLP solution the required data have to be in the french language.

3.2.2 Identifying data source

To extract the required data with the required specification we choose to use the data provided by Moniteur Belge website.

The Moniteur Belge (MB) is the official journal publishing the laws and other regulatory texts of the Belgian State. It owes its name to the Moniteur Universal, the French official journal at the time of the creation of Belgium.
MB ensures the production and distribution of a wide range of official and public publications, both by traditional channel (paper) and by electronic channel (the internet).

The website offers a series of databases, the best known of which is the Belgian Monitor itself, the Morales Annex, and the Auction bulletin , and also takes care of the logistical support for the distribution of a wide range of information brochures published by the Federal Public Service Justice.

3.2.3 Data collection method and tools

Our web source Moniteur Belge website contains a lot of PDF files of the published document by the Belgian State and to collect them we developed a web scraping solution using various tools such as Scrapy and Selenium and based on diffrent features like company ID and the publishing date.

3.2.3.1 Web scraping

Web scraping or web harvesting is the concept of data scraping used for extracting information from a website. The web scraping software can directly approach the World Wide Web by using a web browser or the Hypertext Transfer Protocol.

Even though the web scraping software may be accomplished manually, the concept commonly refers to automated processes accomplished by a bot or web crawler.
It's a way of coping in which certain data is collected and copied from the web, typically into a central local spreadsheet or database, for later retrieval or analysis.

Generally, web pages are created using text-based mark-up languages (HTML and XHTML), and usually a lot of helpful data in text form.
Yet, most web pages are built for human end-users and not for the use of automated use. Therefore, specialized tools and software have been created to help the scraping of web pages.

3.2.3.2 Scrapy

Scrapy[26] is an open-source and free framework for web-crawling developed using Python. In the first place, Scrapy was produced for web scraping, but it enables the developer also to extract data using APIs or to use it as a general-purpose web crawler. At this moment Scrapy is supported by Zyte formerly Scrapinghub, a web-scraping development, and services company.

The project is designed around "spiders", which are self-contained crawlers that start from a set of instructions, and because it follows the idea of other doesn't repeat yourself frameworks, it makes it easier to design and scale large crawling projects by letting the developers reuse their code. In addition, Scrapy presents a web-crawling shell, which allows the developers to test their assumptions on a site's behavior.

3.2.3.3 Selenium

Selenium [27] is a portable framework that can be used to test web applications. It presents a playback tool for authoring functional tests out of the need to master a test scripting language.
In addition, Selenium gives a test domain-specific language for writing tests in a number of one of the most used programming languages like C#, Java, PHP, Groovy, Ruby, Python, Scala, and Perl, and most modern web browsers can run those tests. Selenium runs on Windows, Linux, and macOS and it has four main components Selenium IDE, Selenium RC, Selenium Webdriver, Selenium Grid.

In our project, we used the selenium web diver to perform web scraping, Selenium WebDriver is a web framework that allowed us to execute cross-browser tests. It is used for automating web-based application testing to make sure that it works expectedly and permits selecting a programming language to build test scripts.

3.2.4 Scrapy VS Selenium

Each of the two web scraping tools has its pros and cons. They have similarities and differences as well.

— Scrapy and Selenium are both open-source and free software, which means that their capabilities can be expanded at any time.

— Scrapy is the best performer when it comes to doing complicated roles. Scrapy is the quickest and collects data in record time thanks to its asynchronous system. Selenium, on the other hand, is incapable of handling some data scraping operations.

— Since its execution codes are simple, Selenium is much easier to understand than Scrapy Thanks for a novice in web scraping .

— Scrapy once again outperforms in integrations of proxies and VPNs. It has an adaptable ecosystem. It allocate multiple IP addresses to each request so the user can submit multiple web requests asynchronously. While Selenium will support proxies and VPNs, the process of integrating them is not straightforward.

— When it comes to scraping data from websites using core JavaScript codes, Selenium is the strongest web scraping tool. Scrapy, on the other hand, is more focused on scraping data from HTML and XML formats. For this time being, Selenium does best with concepts such as DOM and can handle AJAX and PJAX requests seamlessly.

3.2.5 Data collection workflows

3.2.5.1 Selenium workflow Architechture

Figure 3.1 [28] displays the Selenium WebDriver workflow Architecture, which is consisted of four major components: the Selenium Client library, the JSON wire protocol via HTTP, the Browser Drivers, and the Browsers.

What occurs in the background when a scraping script is run with WebDriver is for each Selenium Command, an HTTP request is produced and submitted to the browser driver, who then receives the HTTP request via an HTTP server.

An HTTP server determines all of the instructions that must be executed on the browser. Finally, the HTTP server gets the execution status and delivers it back to the automation scripts.

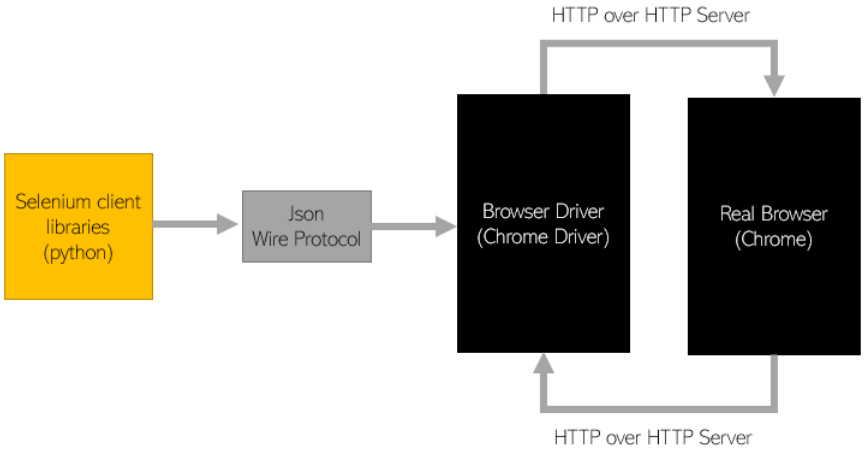


Figure 3.1 – Selenium workflow Architechture

3.2.5.2 Scrapy workflow Architechture

For the scrapy workflow architecture, the execution engine orchestrates and processes the Scrapy data flow. First, the engine receives the original requests to crawl from the spider, then schedules the requests in the scheduler and requests the next requests to crawl.
The scheduler, on the other hand, returns the next Requests to the engine, which transfers the requests to the downloader through the downloader middlewares.

When the page is finished uploading, the downloader generates a response and sends it to the engine via the downloader middlewares. The engine then collects the response from the downloader and sends it to the spider for processing via the spider middleware.

The engine routes processed items to Item pipelines, then routes processed requests to the scheduler and inquires about possible next crawl requests.
In this manner, the procedure is repeated until there are no further scheduler demands. Figure 3.2 depicts this procedure.

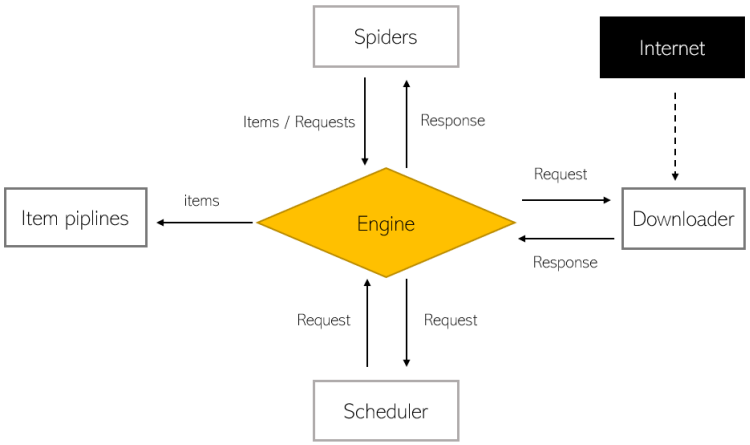


Figure 3.2 – Scrapy workflow Architechture

3.3 Data preparation

3.3.1 Optical character recognition processing (OCR)

3.3.1.1 Definition

Optical character recognition [14] is one of the earliest addressed computer vision and AI applications and it consist of the usage of technology to determine handwritten and printed text characters in a digital image of physical documents, like scanned papers.

The key steps in OCR involve checking the text of a document and translating the characters into code that can be used for data processing. In fact, an OCR system is created with the combination of software and hardware that are used to transform the physical documents into machine-readable text.

The hardware part can be an optical scanner or a specialized circuit board is used to copy or read text,and for the software, part contains a really advanced processing pipeline that may use artificial intelligence to implement more advanced functionalities of intelligent character recognition (ICR) such as determining languages or the handwriting styles.

3.3.1.2 OCR techniques

In the OCR process first, a scanner is used to process the physical document. Then after copying all pages, the OCR software transforms the document into a black and white version. The scanned images are going to be analyzed for dark and light areas, whenever a dark area is identified as a character that has to be recognized, and for the light area is identified as a background.

While the dark area is going to be deeply processed to determine the alphabetic letter or the numeric digit. There is a variety of techniques in OCR programs, however generally implicate targeting one character, word, or block of text at a time.
Then those characters are going to be determined by the implementation of two algorithms:[14]

— **Pattern Recognition:** An OCR model is trained using a variety of formats and fonts which then used to compare and determine the characters in the scanned document.

— **Feature detection:** an OCR program is guided by certain rules regarding the features of a specific letter or number to determine the characters in the scanned document.
Those features may contain the number of angled lines, crossed lines, or curves in the character to compare with. And whenever the character is determined, it is translated to an ASCII code that may be used by the computer for further processing.
The users have to correct basic errors, proofread and be sure that the complex layouts were processed properly before saving the document for future use.

3.3.1.3 OCR processing tool: Tesseract

In our project, we have chosen Tesseract as an OCR tool to process our documents. Tesseract is an open-source software used for optical character recognition in various operating systems. In 2006 it was released as an open-source software sponsored by Google and considered one of the most accurate open-source OCR engines then available.

The reason why we choose tesseract is mainly its powerful features that respond perfectly to the project requirements:

First Tesseract is able to process both left-to-right and right-to-left text as Arabic or Hebrew and a lot of Indic scripts also as CJIK characters like the Chinese, Japanese, and Korean characters.
In addition, It is applicable for use as a back-end and can be used for more complicated OCR tasks including layout analysis.

In its 4th version Tesseract adds a Long short-term memory (LSTM) based OCR engine and models which is an artificial recurrent neural network (RNN) architecture used in the field of deep learning and this for many additional languages and scripts bringing the total to 116 languages.[12]

The Figure 3.3 shows Tesseract's traditional step-by-step pipeline which starts with the input image preprocessing and done with adaptive thresholding where a binary image is generated. Then the connected component analysis is done to give the character outlines.

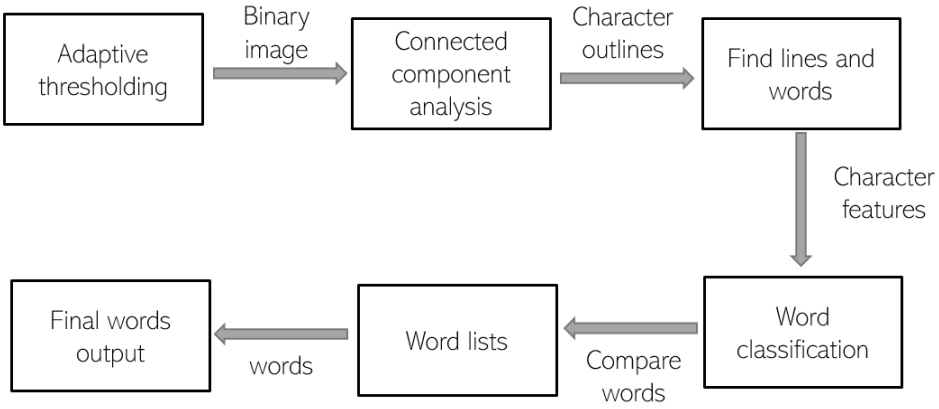


Figure 3.3 – Tesseract OCR engine architecture[17]

Next techniques for character chopping and character association are used to organize the outlines into words.

In the end two-pass word, recognition is done by implementing methods of clustering and classification. For the final decision about the recognized word, Tesseract consults with both the language dictionary and the user-defined dictionary. The word with the smallest distance is provided as an output architecture.

3.3.2 Data annotation

To train models, AI-based solutions require a significant amount of annotated data, which is part of a larger data labeling procedure. A metadata tag is used during the annotation process to indicate up properties of a dataset. With text annotation, the data will includes tags that highlight criteria such as keywords, phrases, or sentences.

In certain applications, text annotation can also include tagging various entities in text, such as 'company name' or 'person name' to teach the machine how to recognize human or

organisation intent behind words. These tags must be comprehensive and precise. Poorly done text annotations will lead a computer to make grammatical mistakes or have difficulties with clarity or context.

3.3.2.1 The different data annotation approaches

The traditional way to get training datasets was to label them manually but these hand-labeled training sets are expensive and time-consuming to create especially when domain expertise is required.
On top of this, tasks often change and evolve in the real world. For all these reasons, practitioners have increasingly been turning to weaker forms of supervision, such as heuristically generating training data with external knowledge bases, patterns, or other classifiers.
To draw the core distinction between these other approaches and weak supervision at a high-level: weak supervision is about leveraging higher-level input from Subject Matter Experts (SMEs).[11]

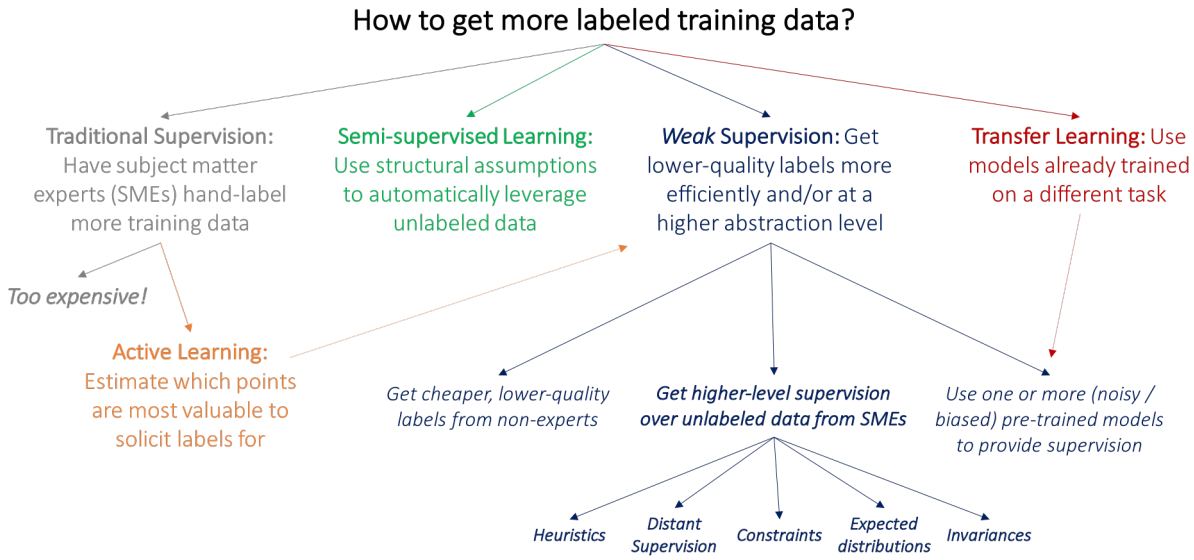


Figure 3.4 – Data Labeling approaches[11]

Figure 3.4 summerizes the different data labeling approaches that we can adapt in get the annotated data that we need.
In **the active learning approach**, the goal is to make use of SMEs more efficiently by having them label data points which are estimated to be most valuable to the model. In the standard supervised learning setting, this means selecting new data points to be labeled. More details about this approach are provided in chapter 2 section 4 .

On the other hand, the goal of **the semi-supervised learning setting** is to use both a small labeled training set and a much larger unlabeled data set. At a high level, we then use assumptions about smoothness, low dimensional structure, or distance metrics to leverage the unlabeled data. Broadly, rather than soliciting more input from SMEs, the idea in semi-supervised learning is to leverage domain and task-agnostic assumptions to exploit the unlabeled data that is often cheaply available in large quantities.

In a typical **transfer learning setting**, the aim is to take one or more models already trained on a different dataset and apply them to our dataset and task. A common transfer learning approach in the deep learning community today is to "pre-train" a model on one large dataset, and then "fine-tune" it on the task of interest. Another related line of work is multi-task learning, where several tasks are learned jointly.

The following paradigms may help us to avoid requesting extra training labels from our SME collaborators. However, the need for labeling some data is unavoidable.[11]

We end up choosing to work with an active learning approach because it requires only weak supervision, the concept is simple to implement and accelerate the annotation process which suites very well what we need in this step.

3.3.2.2 Annotation Tool

What we need for the annotation process in this project is a tool that supports NER labeling, which should be easy to use and provides the data export and import option to be able to perform the active learning process.
To fulfill those requirements, we have chosen the Doccano annotation tool.

Doccano is a human annotation tool. It's an open-source tool that provides annotation features for sequence labeling and text classification labeling in order to create labeled data for named entity recognition, sentiment analysis, text summarization etc.[2]

In addition, it gives the possibility to have it self-hosted which gives more control and the ability to modify the code according to the user's needs.
Pros of Doccano:

— It provides a simple , easy-to-use interface.

— It gives the ability to assign key shortcuts for faster labeling.

— It supports team collaboration functionality.

— It's a free tool.

Cons of Doccano :

— When self-hosted, Doccano shuffles the order of the annotation pieces.

— Laggy and unresponsive at times (in a self-hosted environment).

— No official API. There is an unofficial attempt for an API client.

3.3.2.3 Data labeling guide

Choosing the entity labels is a critical step in the project because it will affect the project result in a significant way. In our project we are interested in extracting main named entities from business documents, in this type of documents the main entities would be the name of companies, people, date and roles so to provide a practical solution we defined the following entities labels :

— **Company Name:** the name of the company producing the document. The name is usually at the beginning of the file in key-value block. They can refer to this company as the "subject company".

— **Legal Rep Last Name:** the last name of each person having a role in this document, for that company.

— **Legal Rep First Name:** the first name of each person having a role in this document, for that company.

— **Function:** each Legal Rep has a function that must be present in the document (to be checked if it is always the case).

— **Valid From :** the start date of the validity of an information, can be written explicitly or referenced to using expressions. For instance, "since the last General Assembly".

— **Valid To:** same as "Valid From", but for the end date.

— **In the name of:** not only humans can have a role in the "subject company", but also a company. In this case, the company allows one of its members to act on its behalf.

The Figure 3.5 shows a screenshot of the entites labeling task using Doccano Application.



Figure 3.5 – Screenshot of entities labeling using Doccano

3.3.2.4 Annotation with Active Learning

In order to perform the active learning process that we described the theory behind in 2.4.2, we chose to adopt the scenario of Pool-based sampling because it's easier to implement when we have an annotation tool like Doccano that does not support files streaming.

We used the uncertainty sampling technique for the querying strategy, and we selected the least confident samples each time because this sampling approach provides the least computation complexity.

Figure 3.6 depicts the active learning cycle that we worked with we start with training a small set of data using a simple model, section 4.5.1 contain the details about the model, after the training if the labeled data that we have is not enough we make a prediction on the unlabeled ones, correct the least certain set and add it to the train set and iterate those steps until we get the amount of labeled data that we want.
In 4.6.2 we present the iterations results and their evaluation.

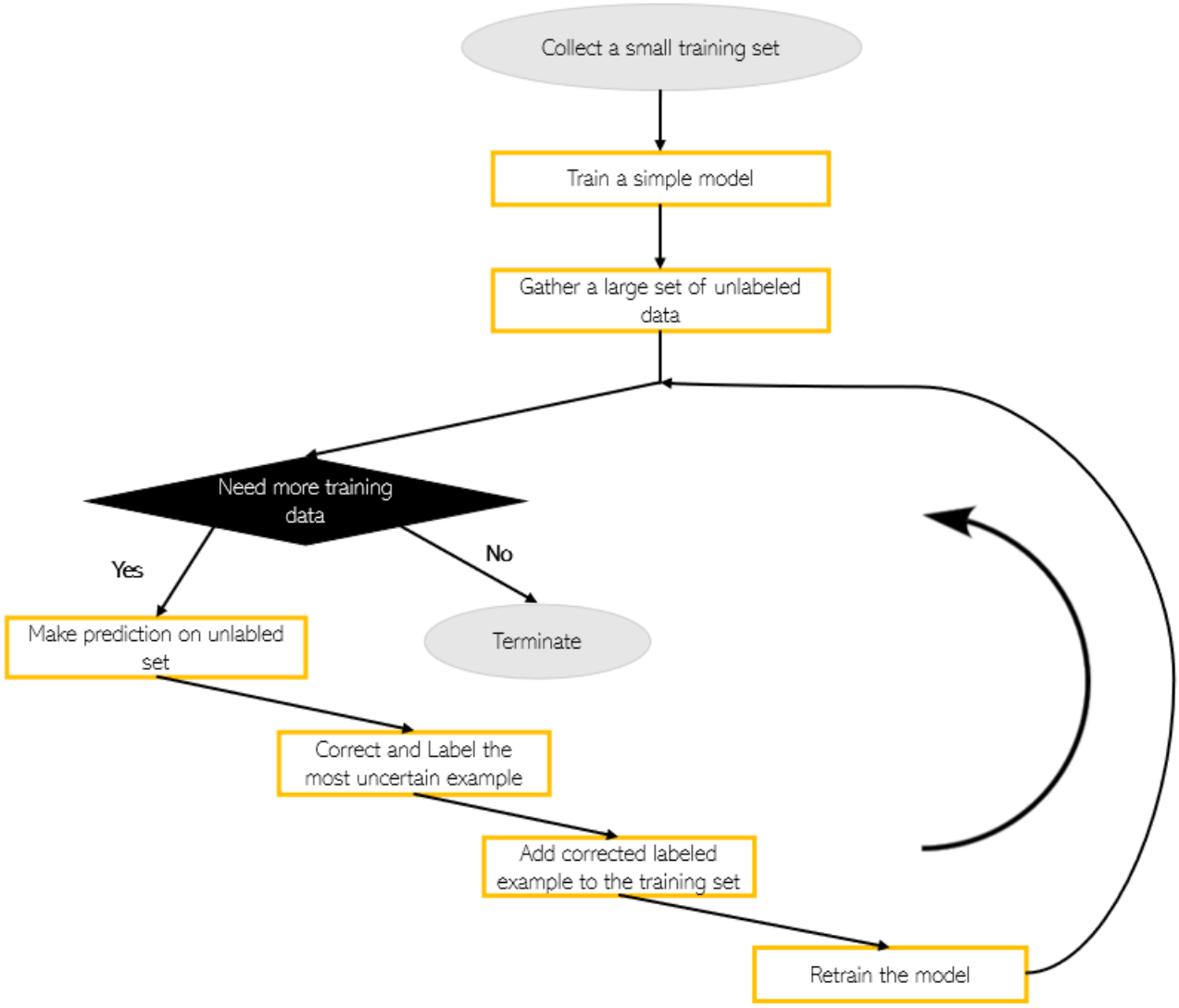


Figure 3.6 – Active learning annotation cycle

3.3.3 Data preparation workflow

The data preparation pipeline requires as input the PDF files from the data that we have collected in the previous task. As the Figure shows the first step in the pipeline is to transform the files from the PDF format to an image format and this because the OCR process can not be executed on a PDF file.

The second step is the OCR process in which we used Tesseract software to transform the images into texts. Since the Moniteur Belge data is mixed some documents are written in french and others in dutch we should filter them using a language detector.

In our project we only interested in the French documents so after the filtering, we kept the Dutch document in storage for future use in other projects and we pushed the french documents to our annotation tool and we start our Active Learning process in a loop as we described in 3.3.2.4 in order to get the annotated data that we need.

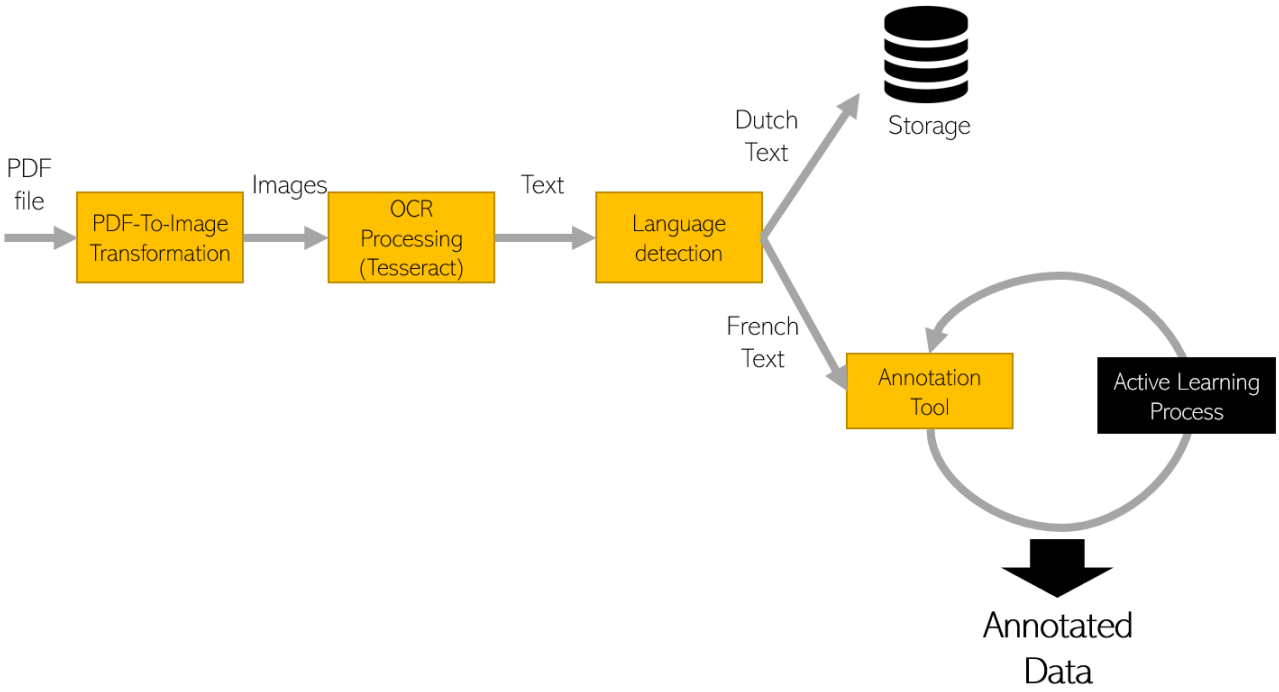


Figure 3.7 – Data preparation pipeline

3.4 Conclusion

This chapter covers the data collection part of our proposed solution. We start the chapter by presenting its goals. Then we justify our choices for the data source as well as the data retrieving mechanisms that we have implemented and its workflows.
After that, we list the different pipeline techniques that we have used to prepare the data.
In the upcoming chapter, we focus on presenting the working environment, the technologies used and the achieved work.

Chapter 4

Solution realization and evaluation

4.1 Introduction

In this chapter, we explain the overall project architecture, which summarizes all of the project tasks and their relationships. In addition, the chapter presents the working environment of the project (libraries, frameworks,etc) and the training process of our models. Moreover, we provide the evaluation metrics and results of each used technique in the project with interpretations.

4.2 Solution workflows: Build & Production

The building project workflow is depicted in Figure 4.1. We began by collecting the required PDF files from the Moniteur Belge website using a web scraping script. The collected files must then be prepared by transforming them into images. After we performed an OCR processing in order to extract texts from those images. Those texts are filtered to keep only the French ones. After that we annotated those texts using the active learning approach that we described in chapter 3 when we finished the annotation the training data was ready to be used for fine-tuning a pre-trained model.

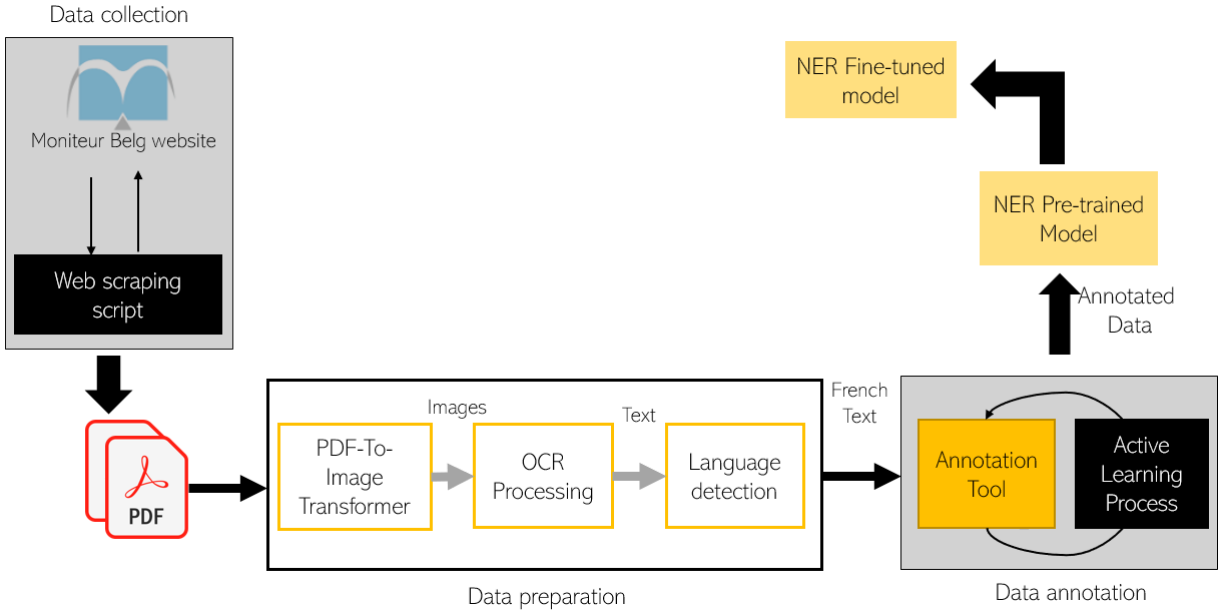


Figure 4.1 – Solution building workflow

Figure 4.2 shows the complete end-to-end solution, in which we begin with a new PDF file and proceed through the same phases as the building data preparation process.
The extracted text from the PDF file will be consumed by the NER fine-tuned model to get the retrieved entities from that text at the end.

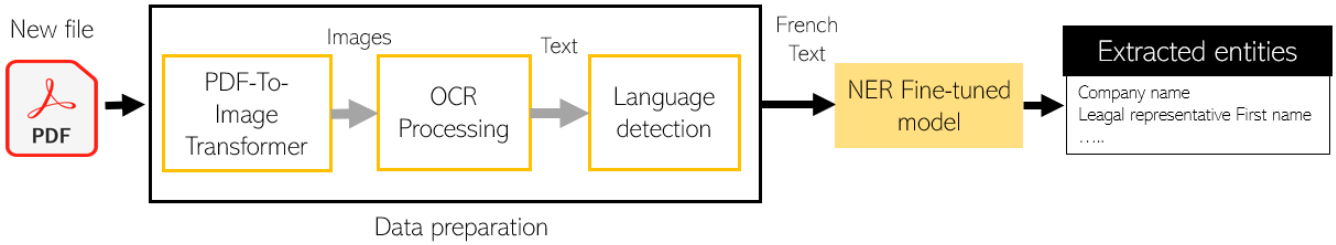


Figure 4.2 – The end-to-end solution workflow

4.3 Setup of the Work Environment

In this section, we detail the specifications of the machines as well as different softwares and technologies used in this work.

4.3.1 Hardware environment

In order to carry out this project, the host company provided us with a laptop computer with the following characteristics:

- **Local Machine**
  - RAM: 12,0 Gb
  - Processor: Intel(R) Core (TM) i5-4300U CPU at 1.9 - 2.49 GHz
  - Operating system: Windows 10

To deploy the Doccano Application and the end-to-end solution application. We used a distant machine with the following characteristics:

- **Virtual Machine : Microsoft Azure Cloud VM**
  - RAM: 32768 MB
  - Processor: 16 cores Intel(R) Xeon(R) Platinum 8168 CPU at 2.70GHz
  - Operating system: Linux (centos 7.8.2003)

4.3.2 Software environment

- **Anaconda:** Anaconda is a free and open source distribution of python and R programming languages for scientific computing such as data science, machine learning applications, large scale data processing, predictive analytics, etc. It integrates the desktop graphical user interface anaconda navigator that allows launching applications. It was used in the project as development environment for most of the data reprocessing scripts.

- **Visual Studio Code:** Is a source-code editor developed by Microsoft for Windows, Linux and macOS. Language extensions can be found and downloaded for free from the VS Code Marketplace. We used Visual Studio Code for web Scraping and for the end-to-end solution development.

— **Google Colab:** an online rich text code editor. It's python notebook feature allows to write, run and test python code on the fly without the need to set up the environment. The code runs on cloud machines, and relieves stress on the machine used for the development which allows for effective computation resources management when developing. Colab was used for the model training and the active learning process.

4.3.3 Frameworks and libraries

During the project, the main library that we worked with was SpaCy because it included the different text processing pipelines and functionalities which helped us to perform the active learning process in addition to models creation training and evaluation.
We used also Pandas , Numpy and Matplotlib mainly for data manipulation and plotting the needed figures for the evaluation. On another hand, we used PyTorch , the Hugging face Library, and Spacy for the Transfer Learning model creation, training, and evaluation.
Streamlit on the other hand was used for the end-to-end application creation because it's a fast tool and so oriented to the machine learning solutions. The following list define each of the mentioned libraries:

— **Matplotlib :**
Matplotlib [5] is a library dedicated to plotting figures with python providing an object oriented API and may make plots in many other toolkits. It gives a lot of methods to customize the plots and change them from a figure to another.

— **Pandas:**
Pandas [7] is an open source library providing high performance and simple way to use and analyze data by python. It is the most used library for analyzing data .

— **NumPy:**
NumPy [6] is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays. NumPy is open-source software and has many contributors.

— **PyTorch:**
PyTorch [8] is an open-source machine learning library based on torch framework and created by Facebook. It's imperative, which means computations happen instantly, so the user doesn't have to wait to examine the code when he's finished. This Python-based library was created to provide flexibility as a deep learning development platform.

— **Hugging face Library :**
Hugging Face [4] is an NLP-focused corporation with a large open-source community, centered on the Transformers library.
Hugging Face Transformers is a Python-based library that exposes an API for using many well-known transformer architectures, such as BERT, RoBERTa or CamemBERT, to achieve state-of-the-art results on a large variety of NLP tasks such as text classification, information extraction, question answering, and text generation.

— **Spacy**
SpaCy [9] is an open-source software library used for advanced NLP applications. It's developed in Python and Cython and it's published under MIT license.

What make Spacy different from the other libraries like NLTK is that spaCy is widely used for research and teaching; it concentrates mainly on providing software for production usage and it supports deep learning workflows as well that make it possible to connect statistical models trained by popular machine learning libraries like Tensorflow, PyTorch or MXNet through its own machine learning library Thinc.

— **Streamlit:** is an application framework for Machine Learning that is open-source and built mainly for Data Science teams.

4.4 Evaluation Metrics

4.4.1 Beam certainty Score

Beam parser is a pipeline component in SpaCy defined as *"beam_ner ()"* that uses the beam search algorithm to measure the certainty of entity prediction.
Beam search is an appealing test-time decoding technique for neural sequence models because it might potentially avoid search errors caused by simpler greedy approaches.
In fact,the beam search allows to estimate probability over items. The probability of an entity is just the sum of the scores of the parses that include it, normalized by the total score provided to all parses in the beam. As a consequence, for each recognized element in the text, the Beam parser assigns a confidence score which the certainty score that we used the active learning process.

4.4.2 Precision

Precision is the ratio of correctly predicted positive observations to the total predicted positive observations. The question that this metric answer is of all reviews that were labeled as positive, how many actually are positive?

$$Precision = \frac{True\ positives}{True\ positives + False\ Positives}$$

Where, True Positives represent a positive instance is classified as positive, if it is classified as negative, it is counted as False Positives .

4.4.3 Recall

Recall is the ratio of correctly predicted positive observations to the all observations in actual class. The question recall answer is: of all reviews that are truly positive, how many did we label?

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

Where, True Negatives represent a negative instance is classified as negative, if it is classified as negative, it is counted as False Negatives .

4.4.4 F Score

F Score is the harmonic mean between precision and recall:

$$F1.Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

This metric takes in consideration both the precision and recall of the model and translates them into one numerical value.

4.5 Modeling

The SpaCy Thinc library is a modular pipeline that provides a number of tools for fast text processing we used two models in our project the first one is the Spacy French Blank Model and the CamemBERT transformer.

The SpaCy french blank model was used mainly in the training during the active learning process because it is a simple model which suites the AL requirements.

Then we used that model one more time by training it with the final training data created and this was just an experiment to compare its performance with CamemBERT later.
On the other hand CamemBERT is mainly used for the final model creation and to laverage the Tansfer Learning techniques that we explained in chapter 2 section 3.

4.5.1 SpaCy French Blank Model

The library parses text to generate a custom spaCy data structure, which is then fed through the pipeline's coordinated components for additional processing.

The pipeline's components are extremely configurable for the fast execution of NLP tasks such as text classification, POS tagging, named entity identification, and so on. Furthermore, these components can be changed independently to adapt to specific implementations.

For our implementation we concentrated on one important component of the spaCy pipeline which is the named entity recognition component. The Spacy component profile is depicted in the Figure 4.3. When we use nlp on a text, it get tokenized to create a Doc object. The Doc is then processed in a series of stages — this is also known as the processing pipeline.

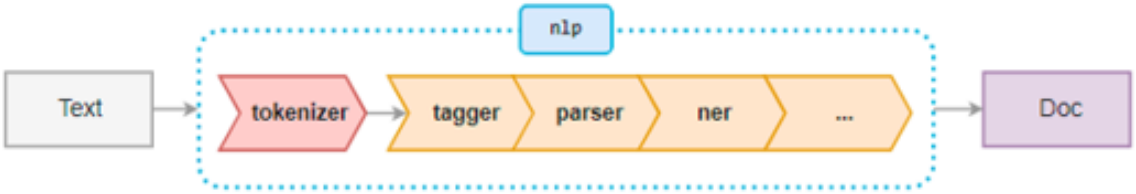


Figure 4.3 – SpaCy NER Architecture

The NER component in SpaCy pipeline is a deep learning model utilizing Convolutional Neural Network (CNN) which we explained its general architecture in part 2.2.4.2. The SpaCy CNN NER Blank model is *spacy.HashEmbedCNN.v2* with the configuration mentioned in Figure 4.4[10]:

After getting the data that we need we tried to train that model using annotated training data to recognize the seven custom entities that we were interested in.

NAME	DESCRIPTION
`width`	The width of the input and output. These are required to be the same, so that residual connections can be used. Recommended values are `96`, `128` or `300`. TYPE: `int`
`depth`	The number of convolutional layers to use. Recommended values are between `2` and `8`. TYPE: `int`
`embed_size`	The number of rows in the hash embedding tables. This can be surprisingly small, due to the use of the hash embeddings. Recommended values are between `2000` and `10000`. TYPE: `int`
`window_size`	The number of tokens on either side to concatenate during the convolutions. The receptive field of the CNN will be `depth * (window_size * 2 + 1)`, so a 4-layer network with a window size of `2` will be sensitive to 17 words at a time. Recommended value is `1`. TYPE: `int`
`maxout_pieces`	The number of pieces to use in the maxout non-linearity. If `1`, the `Mish` non-linearity is used instead. Recommended values are `1` - `3`. TYPE: `int`

```
[model]
@architectures = "spacy.HashEmbedCNN.v2"
width = 96
depth = 4
embed_size = 2000
window_size = 1
maxout_pieces = 3
```

Figure 4.4 – SpaCy Blank NER Model configuration

4.5.2 SpaCy CamemBERT Transformer

SpaCy offers a variety of transfer and multi-task learning procedures that may frequently assist enhance the productivity or accuracy of the solution pipeline.
These approaches may be used to transfer knowledge from raw text into the pipeline, allowing the models to generalize more effectively from annotated instances.

It's possible to load any pretrained transformer model (the background theory of transformers is coverd in part 2.3.4) just by installing spacy-transformers or creating a custom language model for pretraining via the spacy pretrain command. In addition spaCy's transformers provide interaction with PyTorch and the HuggingFace transformers library, providing access to thousands of pre-trained pipeline models.

So we decided to take advantage of this SpaCy's functionality and perform Transfer Learning to add a transformer component to the model pipeline as presented in in figure 4.5.
We choose the spaCy transformer *"camembert-base"* .

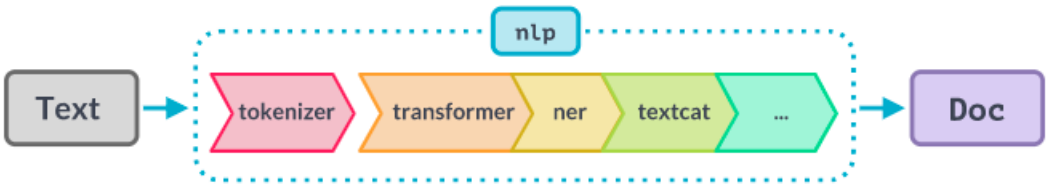


Figure 4.5 – SpaCy NER Architecture with transformer

CamemBERT [21] is a cutting-edge French language model built on the RoBERTa architecture and pretrained on the French sub corpus of the recently released multilingual corpus OSCAR.

It is evaluated for four different downstream tasks for French: part-of-speech (POS) tagging, dependency parsing, named entity recognition (NER), and natural language inference (NLI), outperforming previous monolingual and multilingual approaches for most tasks, confirming the effectiveness of large pretrained language models for French.

Figure 4.6 shows the CamemBERT SpaCy configuration and characteristics that we used.

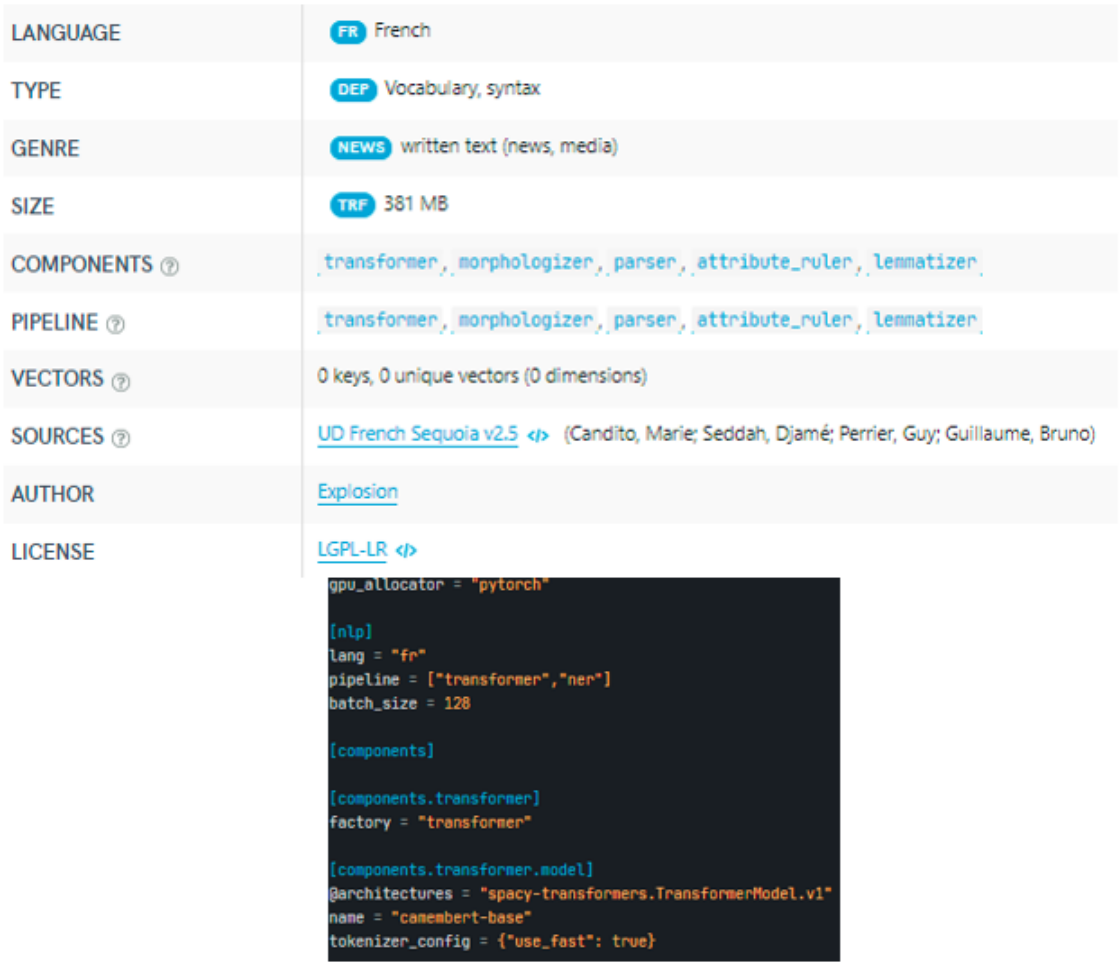


Figure 4.6 – CamemBERT SpaCy configuration and characteristics

4.6 Results and interpretations

4.6.1 Data Collection result

During the data collection process, we were able to use first Scrapy to scrape around 700 documents based on a specific date of publishing, but the files were mixed: 400 were written in French, and the others in Dutch.
In another experiment, we tried to scrape using Selenium, and this time based on the company ID we scraped the published documents by around 300 company which enable us to have 4900 files 3600 of them are French and the rest are in Dutch.

Table 4.1 summarize the data collection result for each method we used and for each language we found.

In our project, we are interested only in the French documents so we used only the french ones and saved the Dutch ones for a future projects.

In Appendix A page we present a sample from the files that we have collected with some highlighted entities in which we are interested in extracting them.

	Total files collected with each method	**French files**	**Dutch files**
Scrapy	700	400	300
Selenium	4900	3600	1300
Total files collected for each language		4000	1600

Table 4.1 – Data Collection Result

4.6.2 Active learning results and model performance through iterations

Following the described active learning approach that we adopted we performed 4 iterations through that process.
Table 4.2 presents the files counts in each iteration in each step and Figure 4.5 presents the model performance in each iteration.

Iterations	**Input Training Files**	**Predicted with the lowest certainty score files**	**Corrected files**
1	100	250	100
2	200	50	37
3	237	184	138
4	375		
Total annotated Data		375	

Table 4.2 – Files count in each Active Learning iteration

— **First iteration:** In the first step we manually annotated 100 documents in order to be able to run the first cycle and after the first training we extracted the 250 documents with the least certainty score from the predicted ones and we corrected 100 of them. This because since we are in the first iteration most of the predicted labels are wrong and the correction process will take so much time so we corrected only 100 documents.

— **Second iteration:** We added the 100 corrected document to the previous training to get 200 documents for training, we trained the model, this time we picked the least accurate 50 documents and corrected them.
In this iteration since most of the prediction have a better result, the correction process was faster most of them needed few adjustments and removing the outliers files so we had 37 corrected document.

— **Third iteration:**in this iteration we had 237 documents for the training set(after adding the corrected set to the previous training set).
We processed another iteration and this time since the prediction is more accurate we picked the 184 with the least confidence score, and we corrected 138 documents of them.

— **Fourth iteration:** This was the final iteration since it gave almost the same performance as the third iteration and we have finally a total training annotated data of 375 training files as we need.

Figure 4.7 depicts the density distribution of the Beam certainty score in each iteration. The files with the least certainty score were selected based on this distribution.

As can be seen in the Figure, the red line curve, representing the fourth iteration, which has the highest density in the area where the certainty score is greater than 0.8 compared to other curves which represent the earlier iterations This demonstrates the performance improvement of the prediction accuracy through active learning iterations.

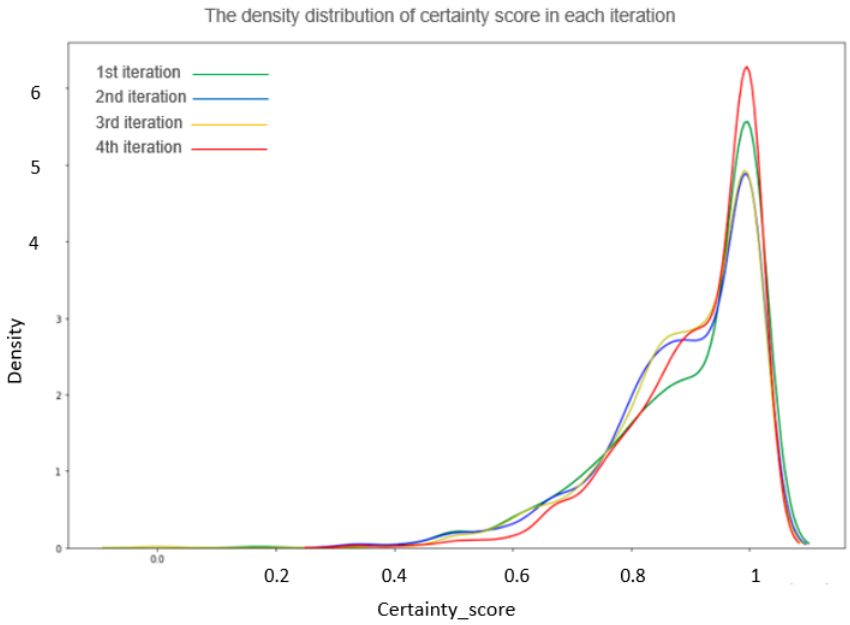


Figure 4.7 – The density distribution of certainty score in each iteration

The iteration's performance improvement is displayed in the Figure 4.8 that depicts the proportion of documents with a successful prediction in each iteration (we defined a good prediction when the Beam certainty score is higher than 0.8).
As we can see, we progressed from 68% of documents with good prediction in the first iteration

to more than 80% of documents with a good certainty score in the fourth iteration.

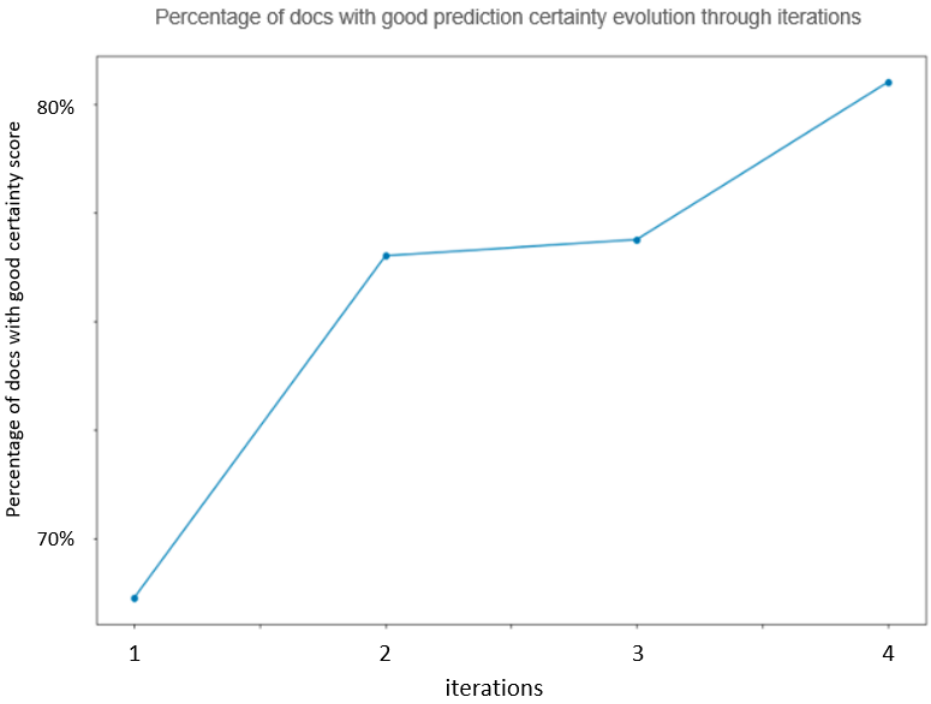


Figure 4.8 – Percentage of docs with good prediction certainty evolution through iterations

Table 4.3 shows the annotated data details after the active learning process, including the count of annotated elements for each label. In total, we were able to annotate 5504 elements, which is sufficient because transfer learning models do not require a large amount of data.

Furthermore, we notice a slight data imbalance because the *"Function"*, *"Legal Representative Last Name"* and *"Legal Representative First Name"* labels are counted more than the others, but this is normal and expected because they are mentioned more in the documents.

In a standard bylaw document, we find the names of many people and their positions, but we usually mention the company's name only once and this is the same for dates and "in the name of" companies.

Label	**Entity Counts**
Function	1122
Legal Representative Last Name	1576
Legal Representative First Name	1560
Company Name	454
Valid From	296
Valid To	358
In Name Of	138
Total	**5504**

Table 4.3 – The Entity Counts of the labled data created with Active Learning

4.6.3 Named entities Recognition models performance

As mentioned in the Modeling section 4.5, for model performance, we first used the Spacy Blank French Model with 70% of our data for training and 30% for testing, and then we used the CamemBERT transformer with the same training data proportion.
Table 4.4 shows the evaluation results for each model, by presenting the model's evaluation metrics values Precision, Recall, and F Score.

Model	**Precision**	**Recall**	**F1 Score**
French Blank SpaCy Model	68.29	63.2	65.65
CamemBERT	75.93	82.14	78.91

Table 4.4 – SpaCy Blank Model & CamemBERT transfomer performance evaluation

The recognition precision shows how many entities were recognized among the already existing ones. The Blank Model precision value is 68.2 while the CamemBERT transformer gives 75.93 and this means that the blank model's model precision is not bad, but the transformer gives a better score.

While the Blank Model's recall value, which characterizes the percentage of relevant results that are correctly recognized by the model, is 63.2, which is quite low and may result in incorrect recognition in some documents, the CamemBERT Transformer presents a very good value of 82.14.

The F Score, which indicates how accurate the model is, gives a 65.65 value in the case of the Blank Model and a 78.91 value when we use the CamemBERT Model, which proves that with the transformer get a better prediction than the Blank model.

All the evaluation metrics that we used to evaluate the performance of each model show that the CamemBERT Transformer Model is more performant and give better result than the French Blank Model and this proves that Transfer Learning techniques have helped to improve the performance Named Entites Recognition task .

4.7 Graphical user interface

In order to finalize our project, we have created a web application with Steamlit to facilitate the visualization of customized named entities extracted from a given text or document.

In the development of this application, we used the default front-end of streamlit and added the following elements.

— **Input text box:** in which the user can type the text that he wants to extract entities from.

— **The Upload File button:** allows the user to upload a DOC, DOCX, PDF, or TXT file from which to extract entities.
The Figure 4.9 shows the Main page elements:

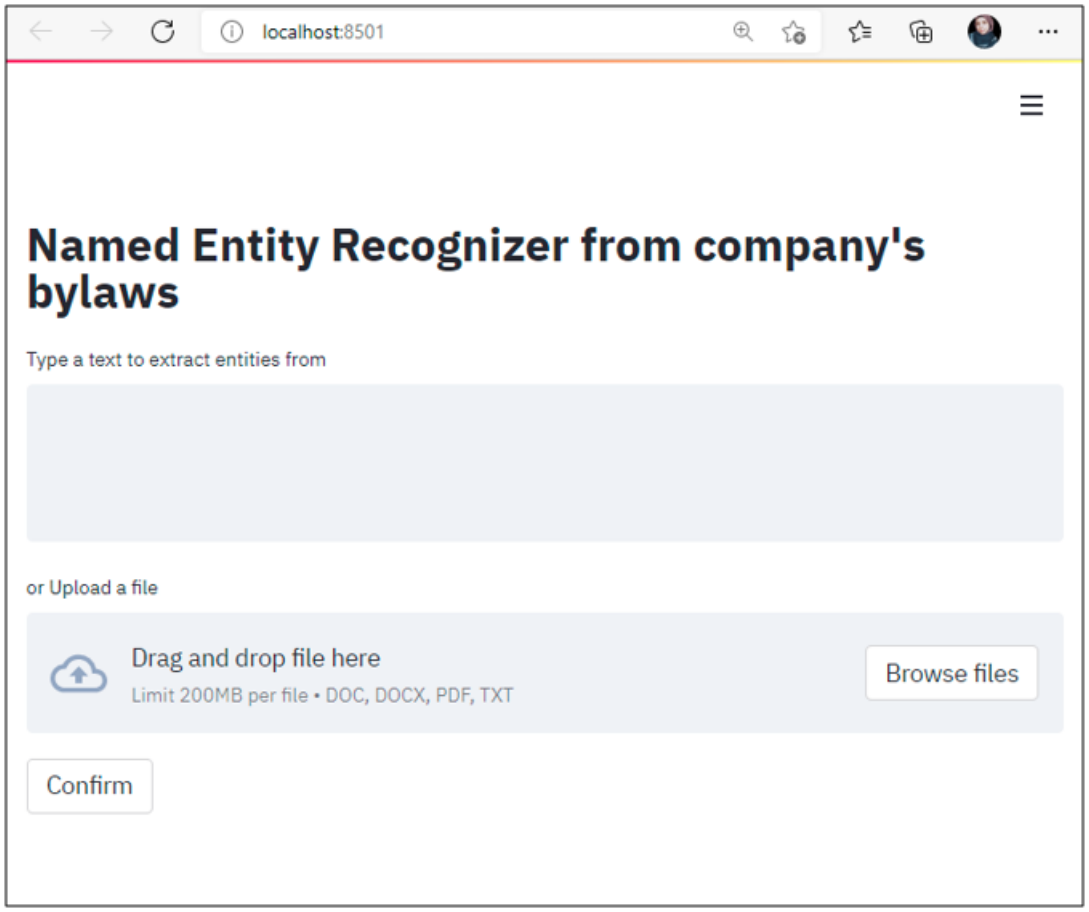


Figure 4.9 – Application Main page

For the back-end process we have two cases :

— If the input is a typed text, DOC, TXT or DOCX file, the input will be pushed directly to the NER model and the text with highlighted extracted entities will be displayed.

— If the file is a PDF, it will go through additional processing, which is the Data preparation processing mentioned in the end-to-end solution pipeline: the file will be transformed

into an image, then to a text using OCR, and pushed to the NER trained model .

Finally the original PDF file page followed by the text with the extracted entities highlighted will be printed.

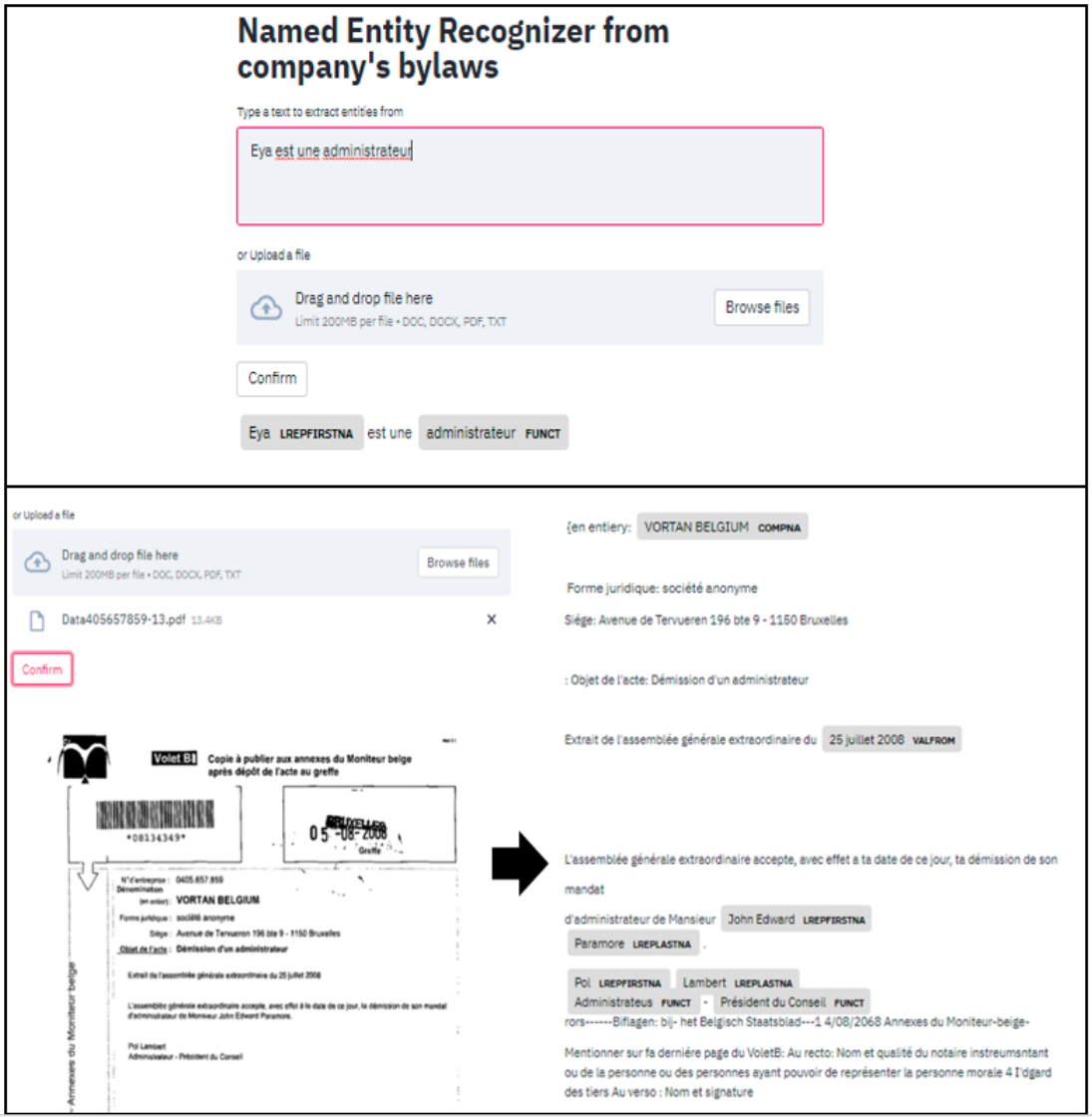


Figure 4.10 – The application use methods

4.8 Project effective realization chronogram

During the realization of our project, we tried to follow the management approach that we described in chapter 1 as well as the time task planning. Figure 4.10 shows the effective project realization chronogram.
We managed to stick to the planning timetable in most of the tasks but it's inevitable that between the tasks time realization and planning there are some differences.

The main difference is concerning the modeling and the data preparation task which we have planned to do them consecutively and we end up overlapping them during a week and this because while the data annotation process we used active learning which is an approach that requires some modeling.

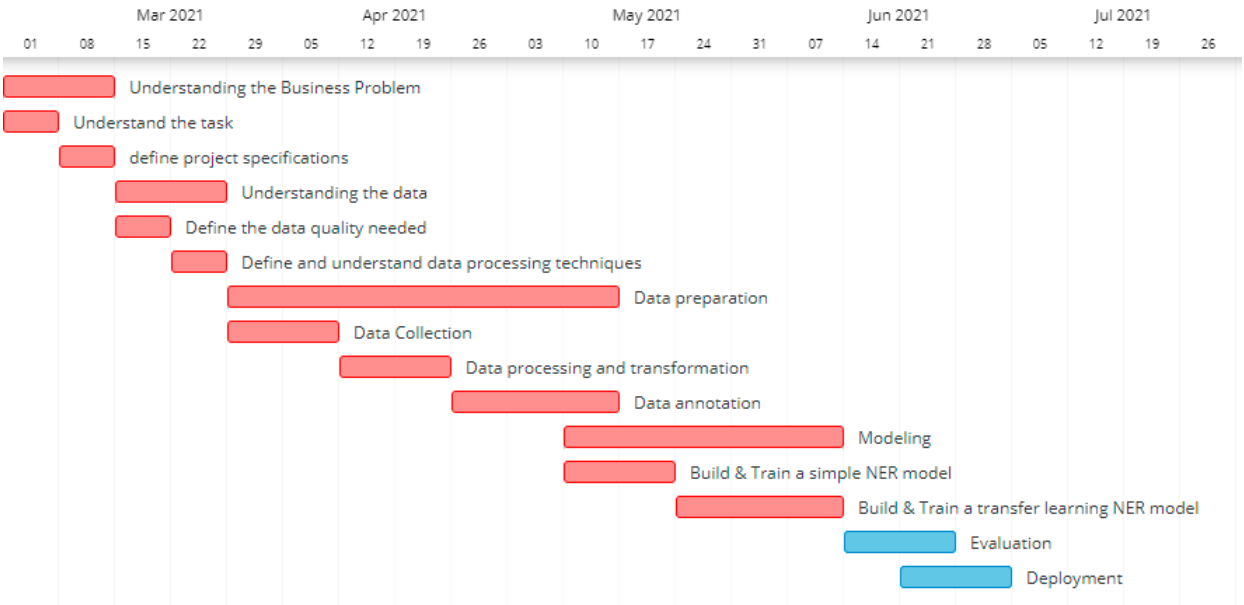


Figure 4.11 – Internship Tasks Realization Time Table

4.9 Conclusion

This chapter has been devoted to the presentation of the global project workflow linking all the tasks that led to the creation of the solution. The chapter also presents the implementation environment and the models used to extract the entities.

Moreover, we evaluated the performance of each technique and interpreted the results. We completed the project by creating a web application that allows us to execute the end-to-end solution to achieve the desired result.

Conclusion and perspectives

This report outlines the work accomplished as part of a graduate internship project. It's titled **Named Entities Extraction from Unstructured Business Documents** and was developed at EY during a four-month period.

We start by presenting our project description, which includes the hosting organism, the problem and the proposed solution main ideas.
Then we focus on the specifying the state of the art AI based techniques in our project. After that, we present the data collection aspect of the project by detailing the collection and preparation approaches.
Finally, we display our achieved work as well as the decisions that went into during the modeling ,evaluation and the end-to-end application development.

It is fair to state that we were able to overcome all of the difficulties that occurred. We were able to provide an additional document processing tool that would help employees to process the company's bylaws and extract the needed information easily and rapidly.
Taking in consideration EY's consulting services, which all turn around helping their clients by offering professional advice and guidance leveraging the recent AI techniques, our solution consists of an additional tool that can be included with EY Document Intelligence Suite.

However, there is yet possibility of adding more improvements. We can work on more data preprocessing and cleaning by adding some rule based techniques to help the model give better results. We can also work towards improving the model result by providing more annotated data through performing more active learning iterations. Moreover, we can also work on adding new labels, merging some of the existing ones or add relation extraction component in the pipeline to make the information extracted from the document more clear.

During this internship, I've had a great opportunity of working and familiarizing myself with state-of -the art technologies, in addition to discovering the fields of NLP and transfer learning. I had the chance, with the support of both my EY supervisor and my university supervisor, to put the knowledge base I gained while studying at Sup'Com in a flourishing and real world work environment.

Bibliography & Netography

[1] Data science project management. www.datascience-pm.com/crisp-dm-2/. [Online; accessed june 2021].

[2] Doccano documentation text annotation for humans. doccano.github.io/doccano/. [Online; accessed june 2021].

[3] Ernst & young website. www.ey.com/en_tn. [Online; accessed june 2021].

[4] Hugging face documentation. huggingface.co/. [Online; accessed june 2021].

[5] Matplotlib documentation. matplotlib.org/stable/contents.html. [Online; accessed june 2021].

[6] Numpy documentation. numpy.org/. [Online; accessed june 2021].

[7] Pandas documentation. https://pandas.pydata.org/docs/. [Online; accessed june 2021].

[8] Pytorch documentation. pytorch.org/. [Online; accessed june 2021].

[9] Spacy documentation. spacy.io/. [Online; accessed june 2021].

[10] Spacy named entities recognizer. docs.scrapy.org/en/latest/. [Online; accessed june 2021].

[11] Braden Hancock Chris Ré Alex Ratner, Paroma Varma and other members of Hazy Lab. Weak supervision: A new programming paradigm for machine learning. ai.stanford.edu/blog/weak-supervision/, 07 2017. [Online; accessed june 2021].

[12] Vaibhaw Singh Chandel. Deep learning based text recognition (ocr) using tesseract and opencv. learnopencv.com/deep-learning-based-text-recognition-ocr-using-tesseract-and-opencv/, 2020. [Online; accessed june 2021].

[13] Minsoo Cho, Jihwan Ha, Chihyun Park, and Sanghyun Park. Combinatorial feature embedding based on cnn and lstm for biomedical named entity recognition. *Journal of biomedical informatics*, 103:103381, March 2020.

[14] TechTarget Contributor. Ocr (optical character recognition). searchcontentmanagement.techtarget.com/definition/OCR-optical-character-recognition. [Online; accessed june 2021].

[15] Jenny Copara, Julien Knafou, Nona Naderi, Claudia Moro, Patrick Ruch, and Douglas Teodoro. Contextualized French Language Models for Biomedical Named Entity Recognition. 2020.

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.

[17] Martin Gjoreski, Gorjan Zajkovski, Aleksandar Bogatinov, Gjorgji Madjarov, Dejan Gjorgjevikj, and Hristijan Gjoreski. Optical character recognition applied on receipts printed in macedonian language. 04 2014.

[18] Beakcheol Jang, Inhwan Kim, and Jong Wook Kim. Word2vec convolutional neural networks for classification of news articles and tweets. *PLoS One*, 14(8), August 2019.

[19] Jing li, Aixin Sun, Ray Han, and Chenliang Li. A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering*, PP:1–1, 03 2020.

[20] Elizabeth D. Liddy. Natural language processing. 2001.

[21] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric de la Clergerie, Djamé Seddah, and Benoît Sagot. CamemBERT: a tasty French language model. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7203–7219, Online, July 2020. Association for Computational Linguistics.

[22] Chris McCormick. Bert fine-tuning tutorial with pytorch. `mccormickml.com/2019/07/22/BERT-fine-tuning/`. [Online; accessed june 2021].

[23] S.J. Pan and Q. Yang. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.

[24] Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar, October 2014. Association for Computational Linguistics.

[25] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.

[26] Scrapy Team. Scrapy documentation. `docs.scrapy.org/en/latest/`. [Online; accessed june 2021].

[27] Selenium Team. Selenium web driver documentation. `www.selenium.dev/documentation/en/`. [Online; accessed june 2021].

[28] Jash Unadkat. Selenium webdriver tutorial : Getting started with test automation. `www.browserstack.com/guide/selenium-webdriver-tutorial`. [Online; accessed june 2021].

[29] Savvas Varsamopoulos, Koen Bertels, and Carmen Garcia Almudever. Comparing neural network based decoders for the surface code. *IEEE Transactions on Computers*, 69(2):300–311, Feb 2020.

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, undefinedukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.

[31] Ramya Vunikili, Vasile Marica, and Dimeji Farri. Clinical ner using spanish bert embeddings. 09 2020.

Appendix A

Here we present a sample of the documents that we have collected in which we highlight manually some of the entities that we want to extract :

